

Data Regression Task Report

목차:

- 1.서론
 - 2.EDA
 - 3.Linear Regression
 - 4.Decision Tree & Random Forest
 - 5.K-Nearest Neighbors
 - 6.결론
-

1.서론

머신러닝 Task중 regression을 시행하였다.

Linear Regression, Random Forest, K-Nearest Neighbors Alg 사용.

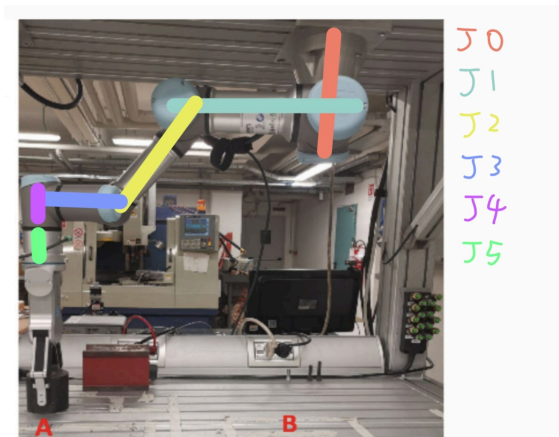
첫 주제로 전류와 속도 데이터로 관절 온도를 예측하려 했으나 상관관계가 유의미하게 보이지 않아 두번째 주제로 그리퍼 구동 전류(Tool Current) 예측으로 회귀 타겟 변경.

Linear Regression으로 Polynomial Features와 Ridge, Lasso, ElasticNet을 각각 적용해 R2, MSE 결과 비교.

2.EDA

주제인 관절별 current와 speed 데이터로 tool current를 예측하기 위해 기본적인 EDA를 진행했다.

2.0.로봇팔



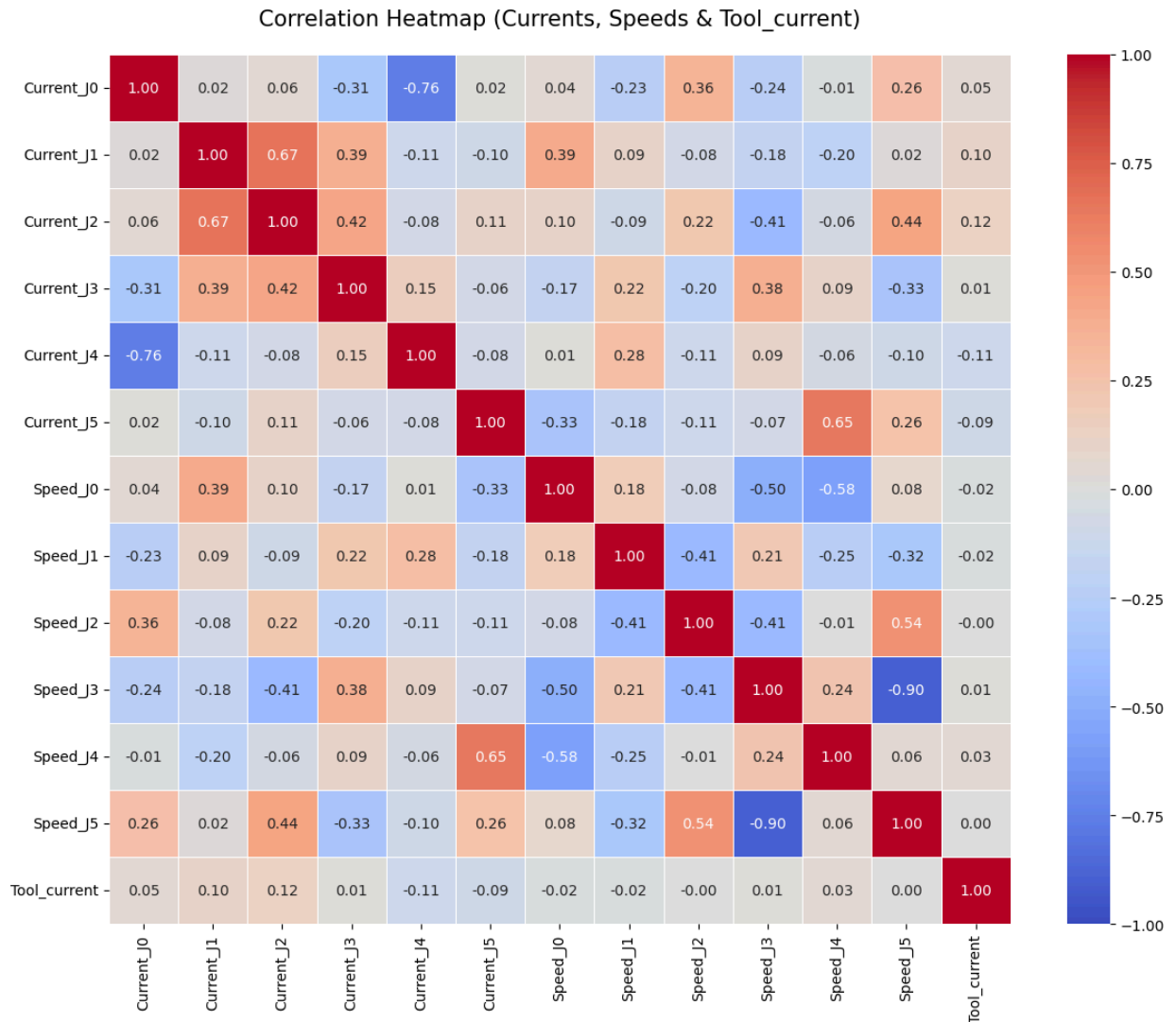
J0은 천장(땅)에 붙어있는쪽으로 다음으로 J1, J2, J3, J4가 오고 J5는 그리퍼의 바로 전 관절이다.

본 연구에서 제공된 실험 벤치 사진을 분석한 결과, 해당 UR3 로봇은 일반적인 지면 거치형이 아닌 상단 알루미늄 프레임에 거꾸로 매달린 천장 거치형(Ceiling-Mounted) 구조로 작동하고 있음을 확인하였다.

이 물리적 거치 환경은 위의 2.5절 페어 플롯과 2.7절 속도 대 전류 분석에서 관찰된 관절 전류 값들이 대다수 음수(-1.5A ~ -3.0A) 대역에 밀집해 있는 현상을 완벽하게 설명해 준다. 지면에 서 있는 로봇과 달리, 천장에 매달린 로봇은 중력에 의해 아래로 처지려는 로봇 팔 자체의 자중을 천장 방향으로 매달아 지탱(Suspension)해야 하므로 버티는 힘(토크)의 방향이 반대로 작용하게 된다.

따라서 관절 전류가 마이너스 대역에서 정적 중력 보상 기동을 형성하는 것은 천장 거치형 로봇이 중력을 거슬러 자세를 유지하기 위한 기계역학적 거동이 데이터 상에 고스란히 반영된 결과임을 알 수 있다.

2.1.전체 변수 히트맵



피어슨 상관계수(Pearson Correlation Coefficient)를 기반으로 작성된 위 히트맵을 통해, 데이터셋의 통계적 특성과 기계역학적 거동에 대한 다음과 같은 주요 정보를 도출할 수 있다.

① 타겟 변수(Tool_current)와 독립변수 간의 극도로 낮은 선형 상관관계

- 그리고 구동 전류인 Tool_current와 다른 모든 관절 변수(전류, 속도)들 간의 상관관계는 최대 0.12 (Current J2)에서 최소 -0.11 (Current J4) 수준에 불과하다.

- 대부분의 관절 속도(Speed) 변수들은 Tool_current와의 상관계수가 0.00 ~ 0.03 근처로 선형적인 연관성이 전혀 존재하지 않는다.

- 시사점: 이는 독립변수와 종속변수 간에 선형적인 비례 관계가 거의 없음을 뜻하며, 단순 다중 선형 회귀(Linear Regression) 모델의 예측력에 한계가 있을 것임을 정량적으로 예견한다. 따라서 비선형적 관계를 포착할 수 있는 다항 회귀(Polynomial Features) 변환이나 의사결정나무 기반 모델(Random Forest, KNN)의 도입이 타당함을 지지하는 근거가 된다.

② 독립변수 간 다중공선성(Multicollinearity) 위험 징후 포착

특정 관절 변수들끼리 매우 강한 선형적 연관성을 보이고 있어, 회귀 모델 설계 시 회귀 계수의 불안정성을 초래하는 다중공선성 문제가 내포되어 있음이 확인된다.

- **Speed J3** 와 **Speed J5 (-0.90)**: 거의 완벽에 가까운 강한 음의 상관관계를 보인다. 이는 로봇 작동 시 말단 손목 관절들이 수평 유지 등을 위해 동조하여 반대 방향으로 회전하는 물리적 제어 메커니즘을 반영한다.

- **Current J0** 와 **Current J4 (-0.76)**: 기저 관절과 손목 관절의 전류 간 강한 반비례 관계가 관찰된다.

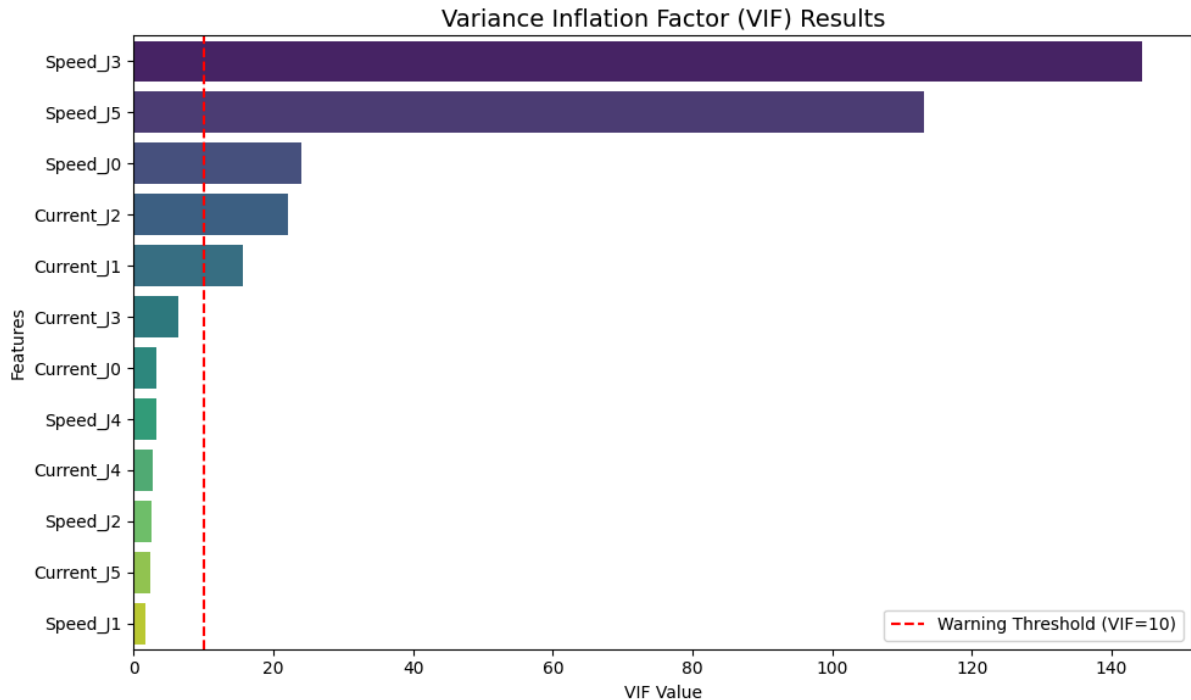
- **Current J1** 와 **Current J2 (0.67)**: 어깨(J1)와 팔꿈치(J2) 관절 전류 간의 유의미한 양의 상관관계는 로봇 팔이 물건의 하중을 들어 올릴 때 두 관절이 물리적 부하를 함께 분담하여 견디는 실제 물리적 현상을 대변한다.

③ 히트맵 분석을 통해 도출한 모델링 전략

- 규제 회귀의 필요성: 독립변수 간의 강한 상관관계(특히 속도 변수들 간의 다중공선성)로 인해 가중치(Coefficient)가 과도하게 팽창할 우려가 있으므로, 가중치를 제어해 주는 **Ridge**, **Lasso**, **ElasticNet** 회귀 모델의 도입이 필수적이다.

- 비선형 모델의 당위성: 타겟과의 선형 상관관계수가 **0.1** 내외로 매우 낮기 때문에, 변수 간의 복잡한 물리적 경계 조건을 스스로 분기하여 학습하는 **Random Forest** 및 **KNN(K-Nearest Neighbors)** 모델이 더 우수한 예측 성능을 보일 것으로 예측된다.

2.2. 다중공선성(VIF, Variance Inflation Factor) 분석



독립변수들 간의 다중공선성을 보다 정량적으로 판별하기 위해 각 변수별 **VIF(분산팽창요인)** 수치를 계산하였다. 통계학적으로 **VIF** 수치가 **10**을 초과하면 해당 변수는 다른 변수들과 심각한 다중공선성을 가짐을 의미하며, 모델의 신뢰성을 저해하는 요인으로 판단한다.

① 임계치(VIF = 10) 초과 변수 리스트

- **Speed J3 (144.29) / Speed J5 (113.16)**: 통계적으로 극심한 수준의 다중공선성이 관찰된다.
- **Speed J0 (24.01) / Current J2 (22.14) / Current J1 (15.60)**: 임계치인 10을 상회하며 다중공선성 경고 범위에 속한다.
- 기타 변수: **Current J3 (6.38)** 이하 나머지 변수들은 모두 10 미만으로 다중공선성 영향권에서 안전한 상태이다.

② 기계공학적 인과 분석 (히트맵 결과와의 교차 검증)

- 손목 관절의 거동 동조성: **Speed J3**와 **Speed J5**에서 발생한 100 이상의 비정상적인 VIF 수치는 앞서 히트맵에서 확인한 두 변수 간의 강력한 음의 상관관계(**-0.90**)를 정량적으로 증명한다.

로봇 손목 관절들이 수평을 유지하며 움직이는 로봇 하드웨어 특성상 두 변수가 물리적으로 긴밀하게 연동되어 있음을 의미한다.

- 대형 관절의 하중 분담: **Current J1**(어깨)과 **Current J2**(팔꿈치)가 동시에 10을 초과한 결과 역시, 로봇 팔의 자체 무게와 작업물 하중을 두 대형 관절이 함께 분담하며 지탱하는 동역학적 특성(상관계수 **0.67**)이 반영된 결과이다.

③ 회귀 모델 성능 개선을 위한 기술적 시사점

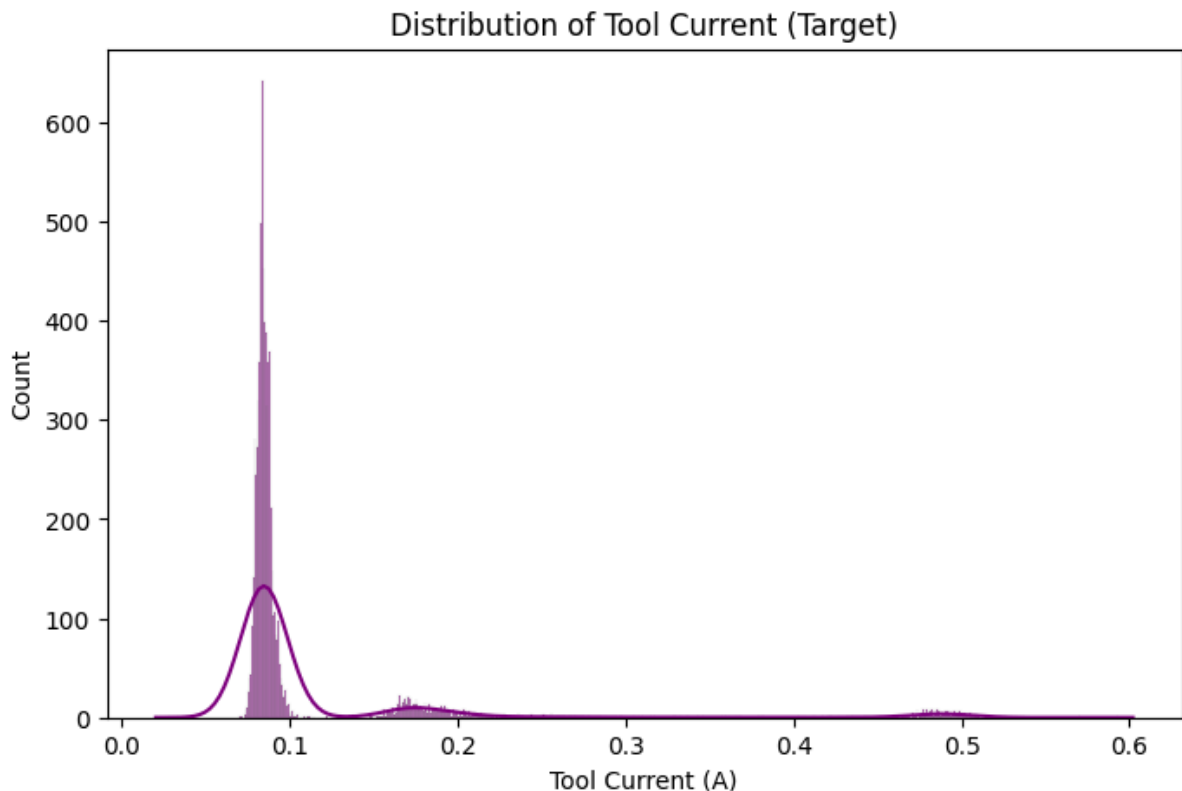
- 기본 선형 회귀(OLS)의 한계: 이처럼 극심한 다중공선성이 존재하는 상태에서 가중치 규제가 없는 일반 선형 회귀를 실행하면, 회귀 계수(Coefficient)의 분산이 비정상적으로 팽창하여 예측 오차가 커지고 모델의 신뢰도가 크게 무너진다.

- 대응 전략 수립:

- 가중치에 패널티를 부여하여 다중공선성 문제를 완화해 주는 규제 회귀 모델(Ridge, Lasso, ElasticNet)의 적용이 필수적이다.

- 혹은 독립변수 간의 상관성을 수학적으로 완전히 차단(상관관계 0)하여 VIF 수치를 1로 수렴시키는 주성분 분석(PCA) 전처리를 교차 적용하는 설계가 이론적으로 강하게 요구됨을 시사한다.

2.3. 타겟 변수(Tool current) 분포 분석



본 회귀 분석의 최종 목표(Target)인 그리퍼 구동 전류(Tool_current)의 데이터 분포와 통계적 특성을 파악하기 위해 히스토그램 및 커널밀도추정(KDE) 그래프를 시각화하였다.

① 삼봉 분포(Tri-modal Distribution) 형태 확인

분석 결과, 데이터는 대칭형 정규분포가 아닌 세 개의 뚜렷한 국소 최댓값(Local Maxima)을 가지는 삼봉 분포의 특성을 보여준다.

- 제1피크 (약 0.08A 부근): 절대다수의 빈도를 차지하는 가장 좁고 높은 피크가 형성되어 있다.
- 제2피크 (약 0.17A 부근): 완만한 높이로 넓게 형성된 중간 영역 피크이다.
- 제3피크 (약 0.49A 부근): 빈도는 극히 낮으나 일정한 범위로 두텁게 형성된 고전류 영역 피크이다.

② 로봇 동작 모드에 따른 물리적 해석

이처럼 세 개의 고유한 전류 대역으로 수렴하는 분산 형태는 그리퍼의 기계적 동작 상태(Operation State)가 고도로 이산화되어 있음을 방증한다.

- 0.08A (대기 및 공운전 구간): 그리퍼가 물건을 잡지 않은 열린 상태로 팔만 이동하거나 대기하는 상태이다. 모터 부하가 없어 최소한의 대기 전류(Standby Current)만 소모하므로 전체 공정 주기 중 가장 높은 비중을 차지한다.

- 0.17A (전환 및 약한 파지 구간): 그리퍼가 쥐어짜는 동작을 시작하거나 가벼운 물건을 전 상태로 모터가 작동하는 전이(Transition) 상태이다.

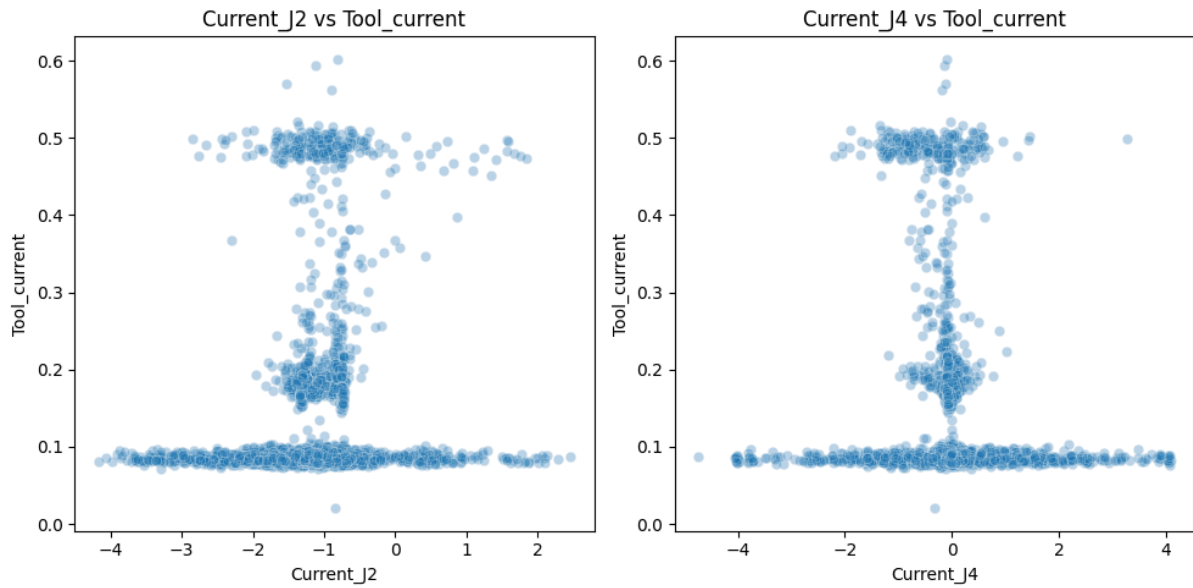
- 0.49A (최대 파지 및 하중 지탱 구간): 그리퍼 모터가 설정된 최대 토크를 발휘하여 가해진 부하(무거운 작업물)를 짊어지고 완벽히 제어하는 고부하 구동 상태이다.

③ 회귀 예측 관점에서의 통계적 한계와 모델 설계 방향성

- 일반 선형 회귀(OLS)의 한계 예견: 타겟 데이터가 이처럼 특정 구간에 고도로 편향되어 분포할 경우, 잔차(Residual)의 정규 분포를 전제로 작동하는 다중 선형 회귀 모델은 예측 왜곡을 겪게 된다. 모델 학습 시 압도적인 빈도를 가진 0.08A 구간의 오차를 줄이는 데만 편향되므로, 실제 고장 진단과 밀접하게 연동되는 중요한 고부하 구간(0.49A)의 예측 성능을 완전히 상실할 위험이 크다.

- 비선형 모델링의 당위성: 따라서 본 회귀 예측 Task에서는 조건문 형식으로 변수의 구간을 쪼개어 단계별 예측을 유연하게 처리하는 의사결정나무 회귀(Decision Tree Regressor) 및 랜덤 포레스트(Random Forest) 모델, 혹은 주변 데이터의 공간 밀집도를 추적하여 예측하는 K-최근접 이웃(KNN) 알고리즘이 물리적·통계적 관점에서 훨씬 정교하고 높은 성능을 보일 것임을 예측한다.

2.4. 주요 관절 전류와 그리퍼 구동 전류 간의 관계 분석



앞서 상관관계 분석에서 비교적 유의미한 수치를 보였던 두 대표 관절 전류 변수인 **Current J2**(팔꿈치) 및 **Current J4**(손목)와 타겟 변수(**Tool current**) 간의 2차원 산점도(**Scatter Plot**)를 시각화하여, 변수 간의 비선형적 관계를 분석하였다.

① 역 T자형(Inverted T-shape) 데이터 분포 확인

- 두 산점도 모두 일반적인 타원형이나 우상향/우하향의 선형 패턴을 따르지 않고, 수평선과 수직선이 직교하여 맞닿은 역 T자형의 독특한 비선형 형태를 지니고 있다.

② 구간별 기계공학적 해석

이와 같은 형태의 데이터 수렴은 로봇의 순차 제어 메커니즘과 동작 특성을 물리적으로 완벽히 설명해 준다.

- 하단 수평 구간 (**Tool_current $\approx 0.08A$**): **Current J2**와 **Current J4**가 각각 **-4**에서 **+4** 영역까지 대역폭 넓게 움직임에도 불구하고 그리퍼 전류(**Tool current**)는 **0.08A** 대기 라인에 완전히 고정되어 있다. 이는 로봇 팔이 역동적으로 움직이더라도 그리퍼가 작동하지 않을 때는 그리퍼 전류가 관절의 거동과 무관하게 완전히 독립적으로 유지됨을 뜻한다.

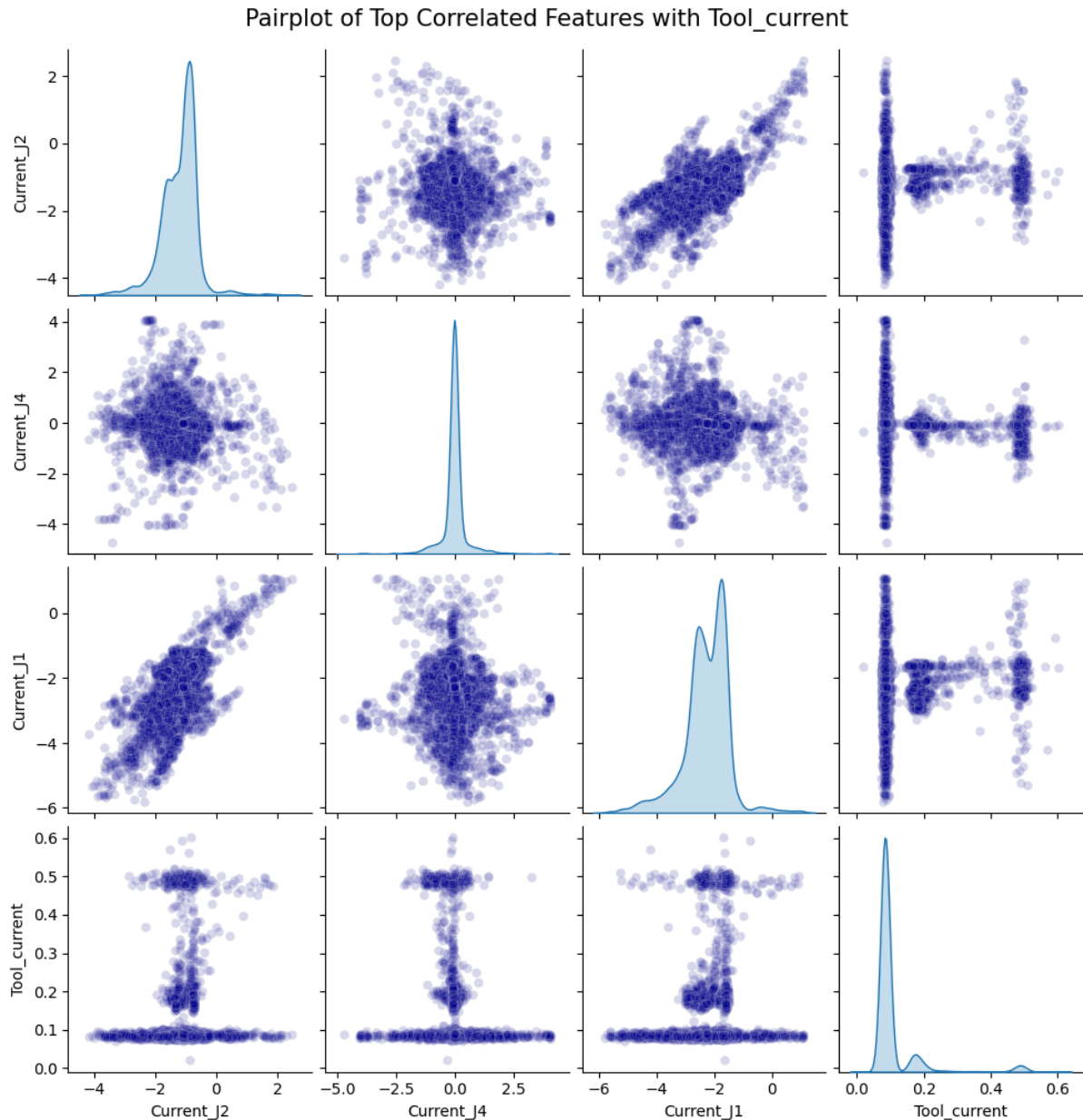
- 수직 기동 구간 (**Current J2 ≈ -1.0 / Current J4 ≈ 0.0**): 관절 전류들이 특정 수치(**Current J2 ≈ -1.0 , Current J4 ≈ 0.0**)로 고정되어 수렴하는 매우 좁은 영역에서만 그리퍼 전류가 수직으로 치솟아 올라간다. 이는 그리퍼가 실제로 구동되는 작업 순간이 로봇 팔이 목적지에 도달해 취하는 특정한 물리적 거동 자세(**Pose**)와 고도로 제어 결합(**Coupled**)되어 작동함을 대변한다.

③ 회귀 모델링에 미치는 영향 및 선형 회귀의 한계

- 선형 모델의 예측 붕괴 예견: 수평 구간과 수직 기둥이 한 지점에서 교차하는 역 T자 분포 하에서는 일차직선 방정식(**Linear Regression**) 기반의 모델이 오차 최소화 연산 과정에서 갈 길을 잃게 된다. 특정 입력 값(예: **Current J2 = -1.0**)에 대해 실제 예측해야 할 타겟 데이터가 **0.08A**부터 **0.6A**까지 다값(**Multi-valued**)으로 분산되어 있어, 선형 모델 적용 시 잔차의 편차가 극대화되기 때문이다.

- 대응 알고리즘 탐색: 이러한 기계적 특징은 가중치의 단순 결합 회귀 방식 대신, '만약 특정 관절 자세 범위에 있으면 그리퍼 전류가 상승하고, 그렇지 않으면 최소 대기 전류로 수렴한다'와 같이 조건부 공간 분할에 특화된 의사결정나무 회귀(**Decision Tree Regressor**) 및 랜덤 포레스트(**Random Forest**) 모델이 본 데이터 예측에 가장 적합한 핵심 해결책이 됨을 강력히 시사한다.

2.5. 주요 결합 변수 페어플롯(Pairplot) 분석



상관분석 결과를 토대로 타겟 변수(**Tool_current**) 및 연관성이 입증된 세 개의 주요 관절 전류 변수(**Current J1**, **Current J2**, **Current J4**) 간의 다차원적인 상관관계를 파악하기 위해 페어플롯(Pairplot)을 시각화하였다. 대각선 영역은 개별 변수의 단변량 커널밀도추정(KDE) 분포를, 비대각선 영역은 두 변수 쌍 간의 이변량 산점도를 나타낸다.

① 대각선 영역(Diagonal)의 단변량 분포 특성 분석

- 어깨와 팔꿈치의 다봉 분포: 하중을 가장 많이 지탱하는 관절들인 **Current J1**과 **Current J2**는 동작 구간별 힘의 부하 상태에 따라 여러 개의 봉우리가 비대칭적으로 편향된 형태의 다봉형 분포를 보여준다.

- 손목 관절의 미세 조정 경향: 손목 관절인 **Current J4**는 **0.0** 부근에 매우 좁고 가파르게 밀집된 단일 봉우리 형태를 나타낸다. 이는 로봇 팔이 구동될 때 손목 관절은 큰 위치

이동보다 중심축 부근에서 중력 균형을 맞추는 미세 자세 조정 위주로 가동되고 있음을 반증한다.

② 어깨(Current J1)와 팔꿈치(Current J2) 관절 간의 강력한 동조 거동

- 본 페어플롯에서 가장 뚜렷하게 관찰되는 변수 쌍 간의 관계는 **Current_J1**과 **Current_J2**가 이루는 격자 산점도이다. 두 관절의 전류 간에는 선명한 우상향 선형 비례 관계(상관계수 **0.67**)가 형성되어 있다.

- 기계 역학적으로 어깨(J1)와 팔꿈치(J2)는 링크의 가속도와 작업물 무게에 따른 물리적 굽힘 모멘트(Bending Moment) 및 회전 하중을 공유하는 핵심 부위이다. 두 센서의 전류가 이처럼 강력하게 실시간으로 연동되는 양상은 두 대형 관절이 물리적 하중 부하를 유기적으로 동시 분담하고 있음을 완벽하게 증명한다.

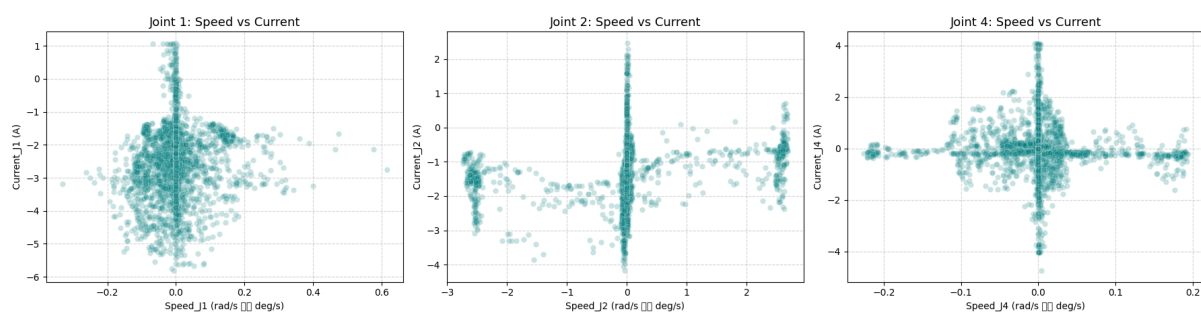
③ 시스템 전반의 물리적 자세 제약성(Global Postural Coupling) 증명

- 최하단 행의 **Tool_current** 산점도들을 비교해 보면, 앞서 분석한 2번 관절뿐만 아니라 어깨 관절인 **Current J1**과 손목 관절인 **Current J4** 역시 그리퍼 전류와 만나며 동일한 역 T자형 분포를 형성하고 있다.

- 이는 그리퍼가 작동하여 소모 전류가 급상승하는 순간(**0.5A** 부근)은 단순히 개별 관절 한두 개의 물리량에 기인하는 현상이 아니며, 어깨, 팔꿈치, 손목이 동기화되어 특정한 가동 자세(Pose)에 도달해 멈추는 로봇 팔 전체 시스템 단위의 정적 제약 상태(Global Postural Constraint)를 시각적으로 증명하는 결정적 근거이다.

2.7. 관절 별 속도(Speed) 및 전류(Current) 간의 동역학적 관계 분석

Mechanical Relationship between Joint Speed and Joint Current



로봇 팔 가동 시 발생하는 기계 역학적 부하 특성을 분석하고 독립변수 간의 물리적 인과성을 규명하기 위해, 로봇의 핵심 3대 가동 영역인 어깨(Joint 1), 팔꿈치(Joint 2), 손목(Joint 4) 관절의 속도(Speed) 대 전류(Current) 산점도를 시각화하였다.

나머지 관절(J0, J3, J5)들은 굳이 그래프로 그리지 않은 이유

J0 (Base): 바닥에서 팽이처럼 수평 회전만 하므로 중력의 영향을 받지 않아 중력 버티기(수직 기동) 현상이 거의 없다. 그래프를 그려봐도 재미있는 물리 현상이 보이지 않습니다.

J3 (Wrist 1) & J5 (Wrist 3): 앞서 VIF 결과에서 확인했듯이, 이 변수들은 각각 VIF가 144와 113으로 극도로 높다. 이것은 J3와 J5가 우리가 뽑은 대표 관절들(J2, J4)과 완전히 똑같이 움직이며 중복된 정보를 보여준다는 뜻이다. 따라서 굳이 똑같은 모양의 그래프를 중복해서 그려서 지면을 낭비할 필요가 없기 때문에 제외했다.

① 속도 0.0 지점 중심의 수직 기동(십자가 형상) 분포 확인

- 세 관절의 그래프 모두 가로축인 속도가 정확히 0.0인 지점을 중심으로 세로축 방향으로 길게 일직선으로 뻗은 독특한 수직 기동이 형성되어 있으며, 결과적으로 전체 데이터가 십자가(Cross) 모양의 특이 분포를 지닌다.

② 기계공학적 해설: 중력 보상(Gravity Compensation) 메커니즘 규명

이 독특한 수직 분포는 로봇 제어 공학의 핵심 이론인 중력 보상(Gravity Compensation) 및 정적 홀딩 토크(Static Holding Torque)의 작용을 물리적으로 완벽하게 입증한다.

- 관절의 속도가 0.0이라는 것은 로봇이 특정 위치에서 완전히 정지해 있음을 의미한다. 그럼에도 불구하고 소모 전류량이 넓은 범위 (최대 $-6A \sim +4A$ $-6A \sim +4A$)로 수직 분산되어 크게 소모되는 이유는 로봇이 정지 상태에서 중력에 의해 아래로 처지려는 힘을 모터 회전력(전류)으로 버텨내고 있기 때문이다.

- 물건을 짊어주고 정지해 있을 때는 높은 전류를 소모하고, 빈 손으로 멈춰 있을 때는 비교적 낮은 전류를 소모하므로 속도가 0인 지점에서 하중(Workload)의 무게에 비례하는 수직 기동 형태의 정적 전류대가 형성된다.

③ 관절 부위별 하중 부담 및 역학적 차이 분석

- 어깨(Joint 1) 및 팔꿈치(Joint 2) 관절: 팔 전체의 링크 자중과 모터 자체 무게를 직접 지탱해야 하는 대형 관절이므로, 실제로 움직이고 있는 동적 구간(속도가 0이 아닐 때)에서도 기저 소모 전류가 각각 약 $-3A$, $-1.5A$ 부근의 높은 음수 영역에 치우쳐 형성된다.

- 손목(Joint 4) 관절: 그리퍼 바로 뒤편에서 자세 제어만 담당하는 가벼운 소형 관절이므로 자체 자중 부하가 적어, 움직이고 있는 동적 구간의 전류 소모가 0.0A 기준선 부근에 매우 조밀하고 안정적으로 수렴하는 가벼운 거동 특성을 보여준다.

④ 회귀 모델 설계 관점에서의 선형 모델의 한계

- 이 십자가형 데이터 분포는 다중 선형 회귀(Linear Regression) 모델의 예측 성능을 무너뜨리는 치명적인 요인으로 작용한다.
- 속도가 0.0인 단 하나의 입력 값에 대해 타겟 전류 값이 수십 배 차이 나는 수직 범위로 분산되어 존재하므로, 최소제곱법(OLS)을 따르는 선형 모델은 이 수직 성분을 전혀 학습하지 못하고 단순히 가로축으로 누워버리는 무의미한 수평 회귀선(기울기 ≈ 0)만을 학습하게 된다.
- 결과적으로 이는 로봇이 가만히 멈춰 서서 그리퍼로 작업을 수행하는 가장 핵심적인 순간(Speed=0)의 전류 예측력을 무력화함을 의미하며, 데이터의 구역을 조건문 형태로 분할하여 예측하는 의사결정나무 회귀(Decision Tree Regressor) 및 랜덤 포레스트(Random Forest) 모델의 채택이 수학적으로 불가피함을 다시 한번 입증하는 강력한 근거이다.

2.8. 탐색적 데이터 분석(EDA) 종합 결론

본 EDA 파트에서 규명한 핵심 통계적·물리적 결론과 이를 바탕으로 수립한 머신러닝 전처리 및 모델링 설계 전략은 다음과 같다.

① 통계적·물리적 한계 요인과 모델 설계의 당위성

- 선형성 부재와 비선형 모델의 정합성: 타겟 변수인 그리퍼 전류(Tool_current)와 관절 센서 변수 간의 선형 상관계수는 최대 0.12 수준으로 극히 미미하며, 산점도 상에서 가로와 세로축이 직교하여 꺾인 역 T자형의 극단적인 비선형 분포를 나타낸다. 따라서 일차방정식 기반의 선형 회귀보다 변수 공간을 다단계 조건 분기로 학습하는 의사결정나무 회귀(Decision Tree Regressor), 랜덤 포레스트(Random Forest), KNN 모델이 통계적·물리적으로 높은 정합성을 가질 것임을 논리적으로 예견한다.
- 심각한 다중공선성 존재와 규제 회귀의 필요성: 기계 구조적 제어로 인해 관절 속도 변수 간의 동조성(Speed J3, Speed J5 간 $VIF > 110$) 및 하중을 분담하는 대형 관절의 전류(Current J1, Current J2 간 $VIF > 15$)에서 극심한 다중공선성이 포착되었다. 일반 선형 회귀는 회귀 계수 팽창으로 신뢰도가 붕괴하므로, 가중치를 패널티로 제어하는 규제 선형 모델과 다항 회귀를 도입하여 상호 비교 검증해야 함을 계량적으로 입증하였다.
- 비정형 삼봉 타겟 분포: 타겟 데이터는 정규분포를 따르지 않고 대기(0.08A), 전이(0.17A), 최대 구동(0.49A) 상태에 따라 세 개의 이산적인 피크에 편향 분산되어 있다. 이는 데이터의 절대다수를 차지하는 대기 상태(0.08A)의 오차 학습에 치우치지 않고, 정밀한 고부하 구동 예측력을 확보할 수 있는 알고리즘 설계가 필요함을 시사한다.

② 전처리(Preprocessing) 기법의 구체화

- 이상치 보존 전략: 전류와 속도가 갑자기 치솟는 임계값 초과 데이터(이상치)는 모션 오작동이나 측정 에러가 아니라, 로봇이 무게 하중을 지탱하는 순간이나 중력 보상 시 발생하는 핵심 물리 신호이므로 제거하지 않고 전원 보존한다.
- 스케일러 선택: 극단적인 이상치로 인해 정상 데이터 분포가 뭉개지는 왜곡을 방지하기 위해, 최댓값에 민감한 **MinMaxScaler**의 사용을 지양하고 데이터 스케일을 고르게 맞추는 **StandardScaler**를 적용하여 성능을 교차 평가하기로 확정한다.

3.Linear Regression

2장에서 확인한 정보를 기반으로 **Current J0~5** 와 **Speed J0~5**를 사용해 **Tool Current**를 예측하는 선형 회귀 코드를 작성하며 알게된 정보와 어째서 이 과정을 하였는지, 고민했던 내용이 무엇인지를 기술한다.

3.1. 순수한 선형회귀, std & mm scaler 적용

--- [최적 피벗] 그리퍼 전류(**Tool_current**) 스케일러별 회귀 결과 ---

[StandardScaler 결과]
 R^2 (결정계수): 0.1047
MSE (평균제곱오차): 0.004917

[MinMaxScaler 결과]
 R^2 (결정계수): 0.1047
MSE (평균제곱오차): 0.004917

아무것도 적용하지 않고 오직 **linear regression**, 그리고 피쳐 스케일링만 했을때의 **R2**는 0.1047, MSE는 0.004917로 각 스케일러의 결과는 동일했다.

3.2. PolynomialFeatures 적용

[MinMax 피쳐 수 변화] 원본: 12개 -> Poly 변환 후: 90개
--- [Poly 적용 후] 그리퍼 전류(**Tool_current**) 스케일러별 회귀 결과 ---

[Standard 결과]
Train R^2 (결정계수): 0.2786 | Train MSE: 0.004611
Test R^2 (결정계수): 0.2340 | Test MSE: 0.004207

[MinMax 결과]
Train R^2 (결정계수): 0.2786 | Train MSE: 0.004611
Test R^2 (결정계수): 0.2340 | Test MSE: 0.004207

Poly를 통해 기존 12개의 피쳐를 2차 다항식으로 변환해 90개의 피쳐를 얻었다.

StandardScaler와 MinMaxScaler 적용 성능 차이는 나타나지 않았으며, 이는 규제가 없는 다중 선형 회귀 모델의 수학적 특성이 반영된 결과이다.

3.3. Poly & 규제(Ridge, Lasso, Elastic Net) 적용

--- 하이퍼파라미터 탐색 중 (잠시만 기다려주세요) ---

--- [최적화 결과] 선형 규제 모델 상위 10개 성능 ---

	Scaler	Model	Alpha	L1_Ratio	R2	MSE
Rank						
1	Standard	Ridge	0.0100	-	0.2340	0.004207
2	Standard	Ridge	0.1000	-	0.2340	0.004207
3	Standard	Ridge	1.0000	-	0.2340	0.004207
4	Standard	Ridge	10.0000	-	0.2340	0.004207
5	Standard	ElasticNet	0.0001	0.500000	0.2334	0.004210
6	Standard	ElasticNet	0.0001	0.200000	0.2334	0.004210
7	Standard	ElasticNet	0.0001	0.800000	0.2325	0.004215
8	Standard	Lasso	0.0001	-	0.2322	0.004217
9	Standard	Ridge	100.0000	-	0.2309	0.004224
10	Standard	ElasticNet	0.0010	0.200000	0.2299	0.004229

규제를 적용하자 각 모델별 R2, MSE 결과를 비교할 수 있었다.

비교 결과 Ridge에서 0.01 ~ 10.0까지의 모델이 0.234와 0.0042로 좋은 결과가 나왔다.

Ridge가 상위권을 점유했다는 것은, 90개로 늘어난 다항 교차 변수들이 특정 몇 개만 중요한 것이 아니라, 자잘한 변수들이 모두 미세하게나마 tool current를 설명하는 데 기여하고 있음을 의미한다. 따라서 변수를 아예 지워버리는 Lasso 방식은 본 데이터의 다항 적합에 불리하게 작용했다는걸 알 수 있다.

3.4. k-fold

--- 하이퍼파라미터 탐색 중 (데이터 셔플링 교차 검증 진행) ---

--- [최적화 완료] 교차 검증 기반 상위 10개 모델의 최종 Test 세트 성능 ---

	Scaler	Model	Alpha	L1_Ratio	CV Train R ²	Test R ²	Test MSE
Rank							
1	MinMax	Ridge	0.0100	-	0.2499	0.2298	0.004230
2	Standard	ElasticNet	0.0001	0.500000	0.2495	0.2334	0.004210
3	Standard	Ridge	10.0000	-	0.2495	0.2340	0.004207
4	Standard	ElasticNet	0.0001	0.800000	0.2495	0.2325	0.004215
5	Standard	ElasticNet	0.0001	0.200000	0.2494	0.2334	0.004210
6	Standard	Lasso	0.0001	-	0.2492	0.2322	0.004217
7	Standard	Ridge	1.0000	-	0.2487	0.2340	0.004207
8	Standard	Ridge	0.1000	-	0.2483	0.2340	0.004207
9	Standard	Ridge	0.0100	-	0.2482	0.2340	0.004207
10	Standard	ElasticNet	0.0010	0.200000	0.2479	0.2299	0.004229

k fold(5 fold) 적용 결과 minmax 스케일러를 제외한 모든 순위가 standard 스케일러로 나왔다.

MinMax 스케일러를 적용한 결과가 1위로 나왔는데 CV Train R2 기준이며 좋은 모델의 관점에서는 적합하지 않다.

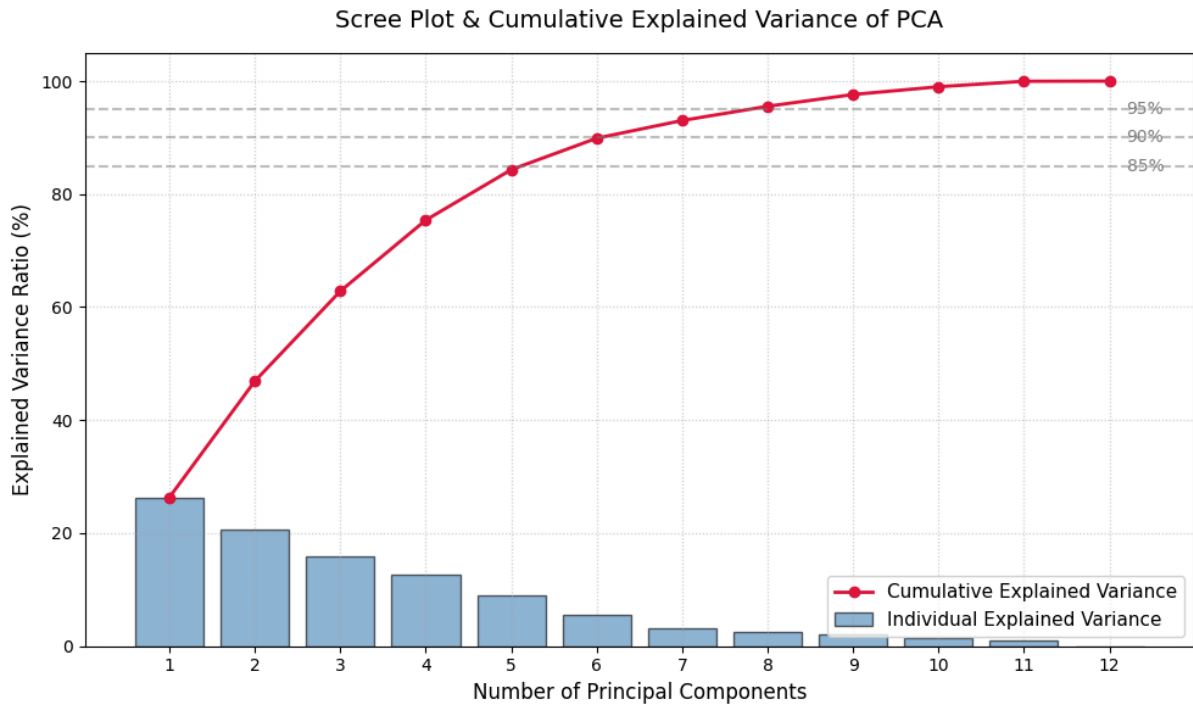
MinMax는 모든 값을 [0, 1] 범위로 압축하고 여기에 degree=2 다항 변환을 하면, 값이 이미 0~1 사이이므로 제곱/교차항이 원본보다 더 작아져 피쳐 간 스케일 차이가 줄어든다.

이 특성이 CV 학습 폴드에서는 미세하게 유리하게 작용했지만, Test 세트에서는 일반화가 덜 되어 즉, 살짝 과적합된 것으로 예상할 수 있다.

MM과 STD를 비교해보면 Train과 Test의 Gap이 0.0201로 가장 크다. (나머지 : 0.0142~0.0155)

따라서 1번 모델은 가장 좋은 모델이 아니며 오히려 이 결과표를 보고 실전에 투입할 단 하나의 최적 모델을 골라야 한다면, 1번이 아니라 Test 성능이 가장 좋고 방어율도 훌륭한 3번 모델이나 7번, 8번 모델이다.

3.5. 데이터 PCA



J0~5까지의 모든 12개의 속도, 전류값을 PCA해본 결과 PC6까지의 합이 89.89%의 설명력을, PC7까지의 합이 93.03%의 설명력을 가진다.

--- [PC 7 차원 축소 버전] Grid Search를 진행 중입니다 ---

--- [PC 8 차원 축소 버전] Grid Search를 진행 중입니다 ---

=== [5단계] PCA (PC 7 vs PC 8) 하이퍼파라미터 최적화 모델 최종 비교 ===

Dimension	Model	Best Hyperparameters	R2 (Test)	MSE	RMSE
PCA (PC 7)	Ridge	{'model__alpha': 100.0}	0.022211	0.005370	0.073279
PCA (PC 7)	Lasso	{'model__alpha': 0.0001}	0.022120	0.005370	0.073283
PCA (PC 7)	ElasticNet	{'model__alpha': 0.001, 'model__l1_ratio': 0.5}	0.021197	0.005375	0.073317
PCA (PC 8)	Ridge	{'model__alpha': 100.0}	0.029237	0.005331	0.073016
PCA (PC 8)	Lasso	{'model__alpha': 0.0001}	0.029115	0.005332	0.073020
PCA (PC 8)	ElasticNet	{'model__alpha': 0.001, 'model__l1_ratio': 0.2}	0.028888	0.005333	0.073029

PC 7 ~ 8 까지 데이터로 각각 회귀해본 결과, PCA를 하기 전보다 값이 훨씬 나빠졌다.

왜 이렇게 나왔나 하면 PCA는 독립변수들 간의 다중공선성을 없애고 데이터의 분산(Variance)을 최대한 보존하는 방향으로 차원을 축소하는 비지도 학습 기법이다.

반면 회귀 모델의 목적은 타겟 변수(y, 그리퍼 전류)와의 상관관계를 찾는 것이다.

12개의 원본 피쳐(관절 전류 6개, 속도 6개)를 7~8개로 압축하는 과정에서, 전체 데이터의 분산은 95% 이상 보존했을지라도 정작 y를 예측하는 데 필수적이었던 미세한 핵심 신호(Signal)가 노이즈로 취급되어 날아가 버렸기 때문에 성능이 대폭락한 것으로 추정된다. 또한 current와 speed는 물리적 단위나 의미가 전혀 다른 도메인 변수이기에 변수간의 의미를 모른채 수학적 분산이 큰 방향으로 축을 찾았기 때문에 고유의 미세한 상호작용은 소실된다.

--- [6단계] Current / Speed 독립 분할 PCA 기반 K-Fold 최적화 시작 ---

[최적 하이퍼파라미터]

```
{'model__alpha': 0.0001, 'model__l1_ratio': 0.2, 'preprocessor__curr_pca__pca__n_components': 5, 'preprocessor__spd_pca__pca__n_components': 5}
```

R2 (Test) : 0.083068

MSE : 0.005036

RMSE : 0.070962

추가적으로 **speed**와 **current**를 분할하여 PCA하여 회귀해본 결과, 통합 PCA에 비해 약 4배 더 좋은 성능을 보였다.

Speed는 **Speed**대로 거시적 속도 패턴을 요약하고, **Current**는 **Current**대로 거시적 힘의 패턴을 요약하도록 분리하여 물리적 특성을 보존했고 회귀 모델이 **speed**와 **current** 간의 물리적 상호작용의 가중치를 학습할 수 있게 되었기 때문으로 추정한다.

이는 이종 데이터에 대한 도메인 지식 기반의 전처리 분리가 차원 축소 기법의 한계를 보완할 수 있음을 시사한다.

4. Decision Tree & Random Forest

4.1 Decision Tree

앞선 선형 회귀 분석을 통해 다항식 변환과 규제를 적용하더라도 데이터 고유의 비선형성과 역 T자형 분포로 인해 예측 성능($R^2 \approx 0.2334$)에 분명한 한계가 있음을 확인하였다.

이에 따라 의사결정나무 모델을 도입하고자 한다.

본 장에서는 트리 기반 알고리즘을 적용하여 선형 모델의 구조적 한계를 보완하고 예측 성능을 개선하고자 시도한 분석 과정을 기술한다.

```

# 1. 교차 검증 (K-Fold) 설정
kf = KFold(n_splits=5, shuffle=True, random_state=42)

# 2. Decision Tree Regressor 기본 모델 생성
dt_reg = DecisionTreeRegressor(random_state=42)

# 3. 튜닝할 하이퍼파라미터 그리드 설정
# - max_depth: 트리의 최대 깊이 (과적합 방지)
# - min_samples_split: 노드를 분할하기 위한 최소 샘플 수
# - min_samples_leaf: 리프 노드가 되기 위한 최소 샘플 수
param_grid = {
    'max_depth': [None, 5, 10, 15, 20],
    'min_samples_split': [2, 10, 20],
    'min_samples_leaf': [1, 5, 10]
}

# 4. GridSearchCV 실행 (R² 기준으로 최적 파라미터 탐색)
print("--- Decision Tree 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 ---")
grid_search = GridSearchCV(
    estimator=dt_reg,
    param_grid=param_grid,
    cv=kf,
    scoring='r2',
    n_jobs=-1,
    verbose=1
)

```

우선 decision tree 모델로 GridSearchCV를 진행해 보았다. 사용된 하이퍼 파라미터는 max depth(트리의 최대 깊이), min samples split (노드 분할을 위한 최소 샘플 수), min_samples_leaf (리프 노드가 되기 위한 최소 샘플 수)이다.

```

--- Decision Tree 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 ---
Fitting 5 folds for each of 45 candidates, totalling 225 fits

[GridSearchCV 결과]
[최적의 하이퍼파라미터 설정]
  • max_depth: 10
  • min_samples_leaf: 5
  • min_samples_split: 20

최고 교차 검증 R² 점수: 0.3882

```

결과를 보았을때 최적의 하이퍼 파라미터 각각 10, 5, 20에서 R2 스코어가 0.3882로 나왔다. 처음 linear 모델보다 0.15 이상 좋아졌으나 아직 잘 설명할 수 있다고 말하기는 힘들다.

```
--- [최종 결과] Decision Tree Regressor ---
```

```
[Train 데이터 평가]
```

```
R2 (결정계수) : 0.6773
```

```
MSE : 0.002062
```

```
RMSE : 0.045414
```

```
MAE : 0.017672
```

```
[Test 데이터 평가]
```

```
R2 (결정계수) : 0.3211
```

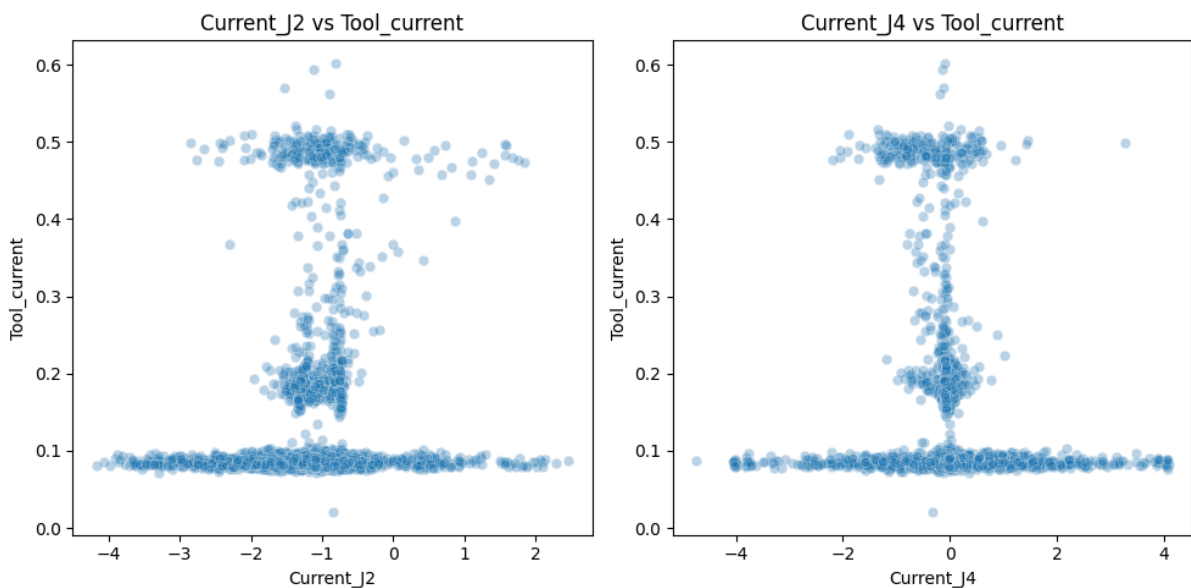
```
MSE : 0.003728
```

```
RMSE : 0.061061
```

```
MAE : 0.023271
```

또한 앞서 도출된 가장 좋은 모델 기준으로 봤을때 **train test**간 차이가 **0.35**정도 나는 심각한 과적합을 보이고 있다.

과적합이 나타난 이유를 추측해보자면, 일단은 관절 전류와 속도만으로는 그리퍼 전류를 완벽히 예측할 수 없는 물리적 한계가 존재한다.



앞서 분석한 **2.4절**의 그래프를 보자면, 가장 그리퍼 전류와 관계가 높았던 **current J2**만 보더라도, 특정 관절 하중을 유지하는 완벽하게 동일한 로봇 팔 자세에서도, 타겟인 그리퍼 전류는 **0.08A**일 수도 있고 **0.49A**일 수도 있다. 입력 데이터가 완전한데 정답이 여러 개인 상황으로 볼 수 있다.

또한 결정나무 모델은 오차를 줄이기 위해 입력 공간을 어떻게든 분할하려는 특성이 존재하기에, 구별 불가능한 데이터들을 억지로 성능을 높이기 위해 맞춘 결과, 깊이를 **10**까지 파고들며 미세한 센서 노이즈나 특정 상황을 암기한 상태이다. 따라서, 훈련

데이터에서는 0.67을 달성했지만, 테스트 데이터에서는 일반화 성능이 0.32로 크게 떨어졌다고 알 수 있다.

게다가 타겟인 그리퍼 전류는 압도적인 타겟 데이터 분포의 불균형을 보여주기에, 탐색된 하이퍼파라미터 중 리프 노드가 되기 위한 최소 샘플 수가 5인것은, 이 데이터의 크기와 불균형을 고려했을때 너무 작다고 볼 수 있다. 트리가 5개의 데이터만 보고 분기해버리니 불필요하게 복잡한 모델이 형성되었다.

해결 방안으로는 현재의 데이터 복잡도에 비해 너무 깊은 트리 깊이를 축소하거나 각각의 최소 샘플수를 소수의 데이터에 맞춰 리프 노드가 생성되는 것을 방지하기 위해 크게 늘리는 방법이 있을것 같다. 타겟 데이터 분포의 불균형이 심하므로 가중치를 부과하여 트리가 중요한 물리적 상태 변화를 우선적으로 학습하도록 유도하면, 불필요한 잔가지가 뺏어 나가는 것을 통제할 수 있을것 같다.

```
# 3. 튜닝할 하이퍼파라미터 그리드 설정
# - max_depth: 트리의 최대 깊이 (과적합 방지)
# - min_samples_split: 노드를 분할하기 위한 최소 샘플 수
# - min_samples_leaf: 리프 노드가 되기 위한 최소 샘플 수
param_grid = {
    # 'max_depth': [None, 5, 10, 15, 20],
    # 'min_samples_split': [2, 10, 20],
    # 'min_samples_leaf': [1, 5, 10]
    # 'max_depth': [4, 5, 6, 7, 8, 9, 10], # 오버피팅을 막기 위해 나무 깊이를 적절히 제한
    # 'min_samples_split': [4, 6, 7, 8, 10],
    # 'min_samples_split': [10, 20, 30],
    # 'min_samples_leaf': [4, 5, 6, 7, 8], # 리프 노드의 최소 샘플 수를 키워 규제 적용
    # 'min_samples_leaf': [5, 10, 15, 20]
    'max_depth': [5, 6, 7, 8],
    'min_samples_leaf': [7, 10, 15, 20, 30], # 리프 노드 제한 범위를 더 넓혀봄
    'min_samples_split': [10, 20, 25, 30]
}

# 4. GridSearchCV 실행 (R² 기준으로 최적 파라미터 탐색)
print("--- Decision Tree 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 ---")
grid_search = GridSearchCV(
    estimator=dt_reg,
    param_grid=param_grid,
    cv=kf,
    scoring='r2',
    n_jobs=-1,
    verbose=1
)
```

하이퍼 파라미터 조정 결과,

```
--- Decision Tree 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 ---
Fitting 5 folds for each of 80 candidates, totalling 400 fits

[GridSearchCV 결과]
[최적의 하이퍼파라미터 설정]
  • max_depth: 7
  • min_samples_leaf: 7
  • min_samples_split: 10

최고 교차 검증 R2 점수: 0.3987
```

최대 깊이는 7, 각 최소 샘플 수는 7과 10으로 나왔다.

```
--- [모델 비교] 상위 5개 Decision Tree 성능 종합 ---
```

	순위	CV R ²	Train R ²	Test R ²	과적합 Gap	Train MSE	Test MSE	파라미터 설정
0	1	0.3987	0.5541	0.3665	0.1876	0.002850	0.003479	D=7, L=7, S=10
1	2	0.3985	0.5521	0.3669	0.1853	0.002863	0.003477	D=7, L=7, S=20
2	3	0.3953	0.6004	0.3430	0.2574	0.002554	0.003608	D=8, L=7, S=20
3	4	0.3931	0.6048	0.3493	0.2555	0.002526	0.003574	D=8, L=7, S=10
4	5	0.3855	0.5525	0.3292	0.2233	0.002860	0.003684	D=8, L=15, S=10

```
=====
--- [상세 분석] 1위 최적 모델 최종 평가지표 ---
=====
```

```
[Train 데이터 평가]
R2 (결정계수) : 0.5541
MSE : 0.002850
RMSE : 0.053385
MAE : 0.023017
```

```
[Test 데이터 평가]
R2 (결정계수) : 0.3665
MSE : 0.003479
RMSE : 0.058985
MAE : 0.025088
```

상위 5개의 성능을 종합해보니, 1위의 Test R² 점수가 원래 0.3211에서 0.3665로 약 4.5%가 상승 하였고 train test R²값의 차이가 0.35 수준에서 0.1876로 절반 가까이 감소하는 효과가 있었다.

더불어, 이 모델에서 실제 테스트 데이터 평가 결과와 과적합 정도를 비교해 보면 2위 모델이 오히려 더 우수하다. 왜냐하면 **test R2**가 1위보다 높고, 과적합 **gap**이 낮으며, **test MSE**는 1위보다 낮기 때문이다. **min samples split(S)** 제한을 10에서 20으로 더 강하게 규제해 주면서, 노드가 너무 잘게 쪼개지는 것을 막아 실전 데이터에서 훨씬 융통성 있게 예측하는 모델이 완성되었다.

```
sample_weights = np.where(y_train >= 0.2, 4.0, 1.0)

# 4. GridSearchCV 실행 (R² 기준으로 최적 파라미터 탐색)
print("--- Decision Tree 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 ---")
grid_search = GridSearchCV(
    estimator=dt_reg,
    param_grid=param_grid,
    cv=kf,
    scoring='r2',
    n_jobs=-1,
    verbose=1
)
```

```
grid_search.fit(X_train, y_train, sample_weight=sample_weights)
```

```
# 5. 최적 모델 및 결과 확인
```

```
best_dt = grid_search.best_estimator_
```

```
print("\n[GridSearchCV 결과]")
```

```
print("[최적의 하이퍼파라미터 설정]")
```

```
for param, value in grid_search.best_params_.items():
```

```
    print(f"    • {param}: {value}")
```

```
print("")
```

```
print(f"최고 교차 검증 R² 점수: {grid_search.best_score_:.4f}")
```

✓ 0.9s

```
--- Decision Tree 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 ---
```

```
Fitting 5 folds for each of 80 candidates, totalling 400 fits
```

```
[GridSearchCV 결과]
```

```
[최적의 하이퍼파라미터 설정]
```

- max_depth: 7
- min_samples_leaf: 20
- min_samples_split: 10

```
최고 교차 검증 R² 점수: 0.4968
```

추가적으로 0.2A 이상의 고부하 전류(0.49A 부근)에 가중치를 부여하고 학습 과정에 적용하였다.

--- [모델 비교] 상위 5개 Decision Tree 성능 종합 ---

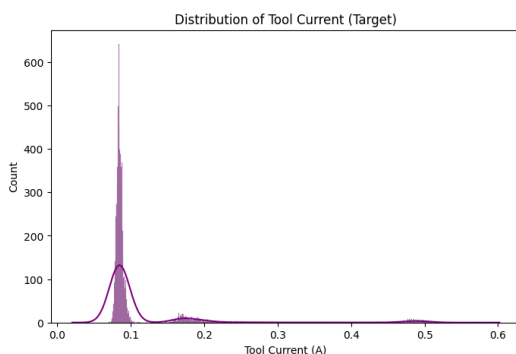
	순위	CV R ²	Train R ²	Test R ²	과적합 Gap	Train MSE	Test MSE	파라미터 설정
0	1	0.4968	0.4982	0.3130	0.1852	0.003208	0.003773	D=7, L=20, S=30
1	1	0.4968	0.4982	0.3130	0.1852	0.003208	0.003773	D=7, L=20, S=25
2	1	0.4968	0.4982	0.3130	0.1852	0.003208	0.003773	D=7, L=20, S=20
3	1	0.4968	0.4982	0.3130	0.1852	0.003208	0.003773	D=7, L=20, S=10
4	5	0.4949	0.5295	0.2999	0.2296	0.003007	0.003845	D=8, L=20, S=10

--- [상세 분석] 1위 최적 모델 최종 평가지표 ---

[Train 데이터 평가]
R² (결정계수) : 0.4447
MSE : 0.003549
RMSE : 0.059575
MAE : 0.028991

[Test 데이터 평가]
R² (결정계수) : 0.1099
MSE : 0.004888
RMSE : 0.069917
MAE : 0.032798

방금 가장 좋았던 2번 모델에 비해 과적합 gap이 0.001 줄어들긴 하였으나, test R2가 0.0539 줄어들었으며, MSE 값도 0.0296 늘어나 예측 성능이 떨어지게 되었다.



어째서 이렇게 나왔는지 예측하자면, 2.3절에서 봤듯이, 전체 데이터의 80% 이상이 0.08A 근처에 몰려 있다. 우리가 평가 기준으로 쓰는 일반 R2 점수는 개수가 많은 데이터를 틀렸을 때 감점을 가장 크게 하도록 설계되어 있다. 즉, 여기에 소수 데이터에 가중치를 주면, 모델은 0.49A를 맞추기 위해 대다수인 0.08A 구간의 예측력을 일부 희생하게 되어, 결과적으로 개수가 압도적으로 많은 0.08A 쪽에서 오차가 발생하고, 이것이 전체 test R2 점수를 폭락하게 만든 것 같다.

```
# 2. 결측치 없는 클린 데이터 분리
df_clean = df.dropna().copy()
# -----
# [추가됨] 2.5 도메인 지식 기반 파생 변수 (Feature Engineering)
# 로봇이 멈춰있는 상태(Speed=0)를 트리가 명시적으로 알 수 있도록 이진 변수 추가
# 미세한 센서 노이즈를 고려하여, 6개 관절 속도 절대값의 합이 0.05 미만이면 정지(1)로 간주
# -----
speed_cols = ['Speed_J0', 'Speed_J1', 'Speed_J2', 'Speed_J3', 'Speed_J4', 'Speed_J5']
df_clean['is_stationary'] = (df_clean[speed_cols].abs().sum(axis=1) < 0.05).astype(int)
# 3. 입력 피처(x) 및 타겟(y) 설정 (파생 변수 포함)
input_cols = [
    'Current_J0', 'Current_J1', 'Current_J2', 'Current_J3', 'Current_J4', 'Current_J5',
    'Speed_J0', 'Speed_J1', 'Speed_J2', 'Speed_J3', 'Speed_J4', 'Speed_J5',
    'is_stationary' # 새롭게 만든 파생 변수를 학습 피처로 추가
]
X = df_clean[input_cols]
y = df_clean['Tool_current']
print(f"데이터 크기: {df_clean.shape}")
print(f"피처 개수: {len(input_cols)}개, 타겟: Tool_current")
✓ 0.0s

데이터 크기: (7355, 25)
피처 개수: 13개, 타겟: Tool_current
```

EDA에서 발견한 속도가 0.0인 지점의 수직 기동 특성을 살려, 6개의 관절 속도의 절대값의 합이 모두 0에 가깝다면 True(1), 아니라면 False(0) 이라는 정지 여부 파생 변수를 Feature Engineering 해보았다

이렇게 하면 트리가 여러 노드를 거치며 힘들게 속도가 0인지를 찾아내는 대신, 최상단 루트 노드에서 한 번에 가장 중요한 기계적 상태를 분기할 수 있어 트리의 구조가 단순해지고 과적합이 줄어들 것으로 예상되었다.

--- [모델 비교] 상위 10개 Decision Tree 성능 종합 ---

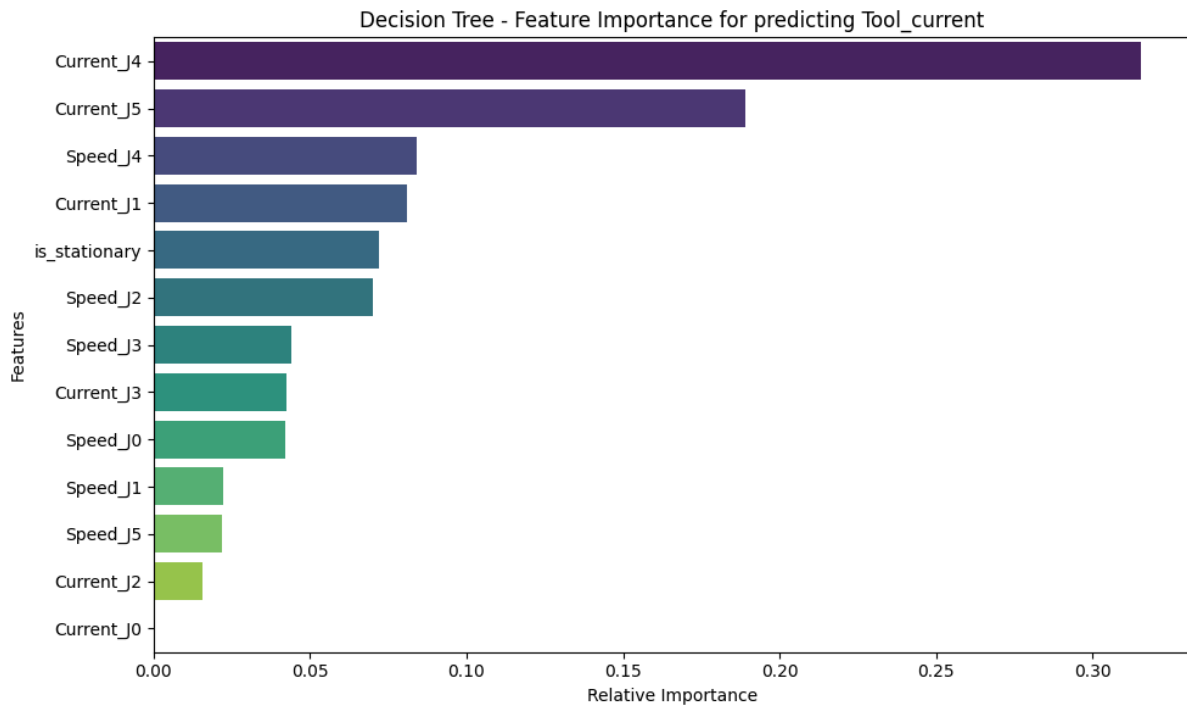
	순위	CV R ²	Train R ²	Test R ²	과적합 Gap	Train MSE	Test MSE	파라미터 설정
0	1	0.4256	0.6253	0.4442	0.1810	0.002395	0.003052	D=8, L=7, S=20
1	2	0.4192	0.5163	0.3624	0.1539	0.003092	0.003501	D=8, L=30, S=25
2	2	0.4192	0.5163	0.3624	0.1539	0.003092	0.003501	D=8, L=30, S=20
3	2	0.4192	0.5163	0.3624	0.1539	0.003092	0.003501	D=8, L=30, S=10
4	2	0.4192	0.5163	0.3624	0.1539	0.003092	0.003501	D=8, L=30, S=30
5	6	0.4169	0.6397	0.4697	0.1700	0.002303	0.002912	D=8, L=7, S=10
6	7	0.4168	0.5990	0.4201	0.1789	0.002563	0.003185	D=7, L=7, S=20
7	8	0.4164	0.5567	0.4273	0.1294	0.002833	0.003145	D=6, L=7, S=20
8	9	0.4150	0.5586	0.4342	0.1244	0.002821	0.003108	D=6, L=7, S=10
9	10	0.4149	0.6089	0.4391	0.1698	0.002500	0.003081	D=7, L=7, S=10

실제로 모델을 돌려보니 피쳐 엔지니어링 도입 전 최고 성능의

Test R2 : 0.3669, 과적합 Gap : 0.1853 , Test MSE : 0.003477 보다

피쳐 엔지니어링 도입 후 최고 성능인 6위 모델을 비교하면

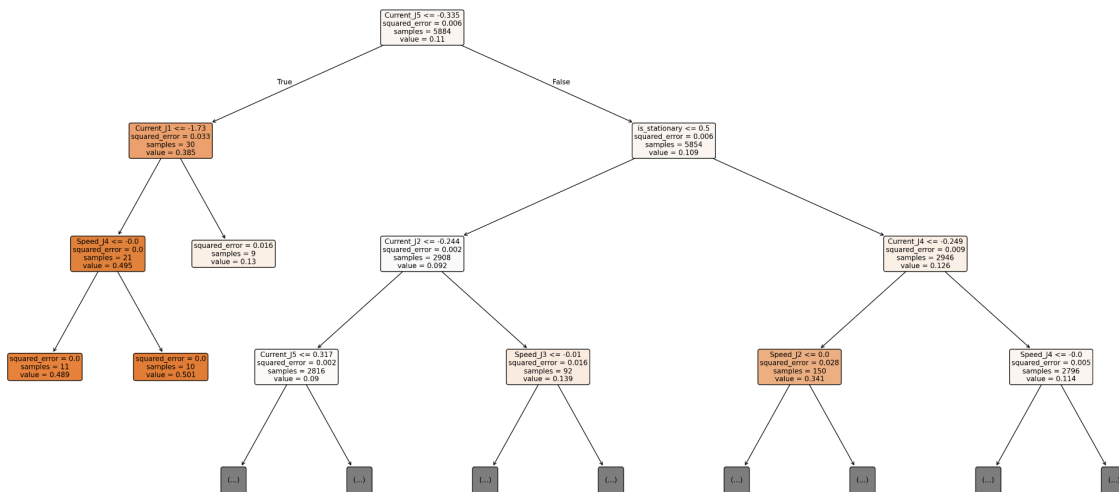
Test R2 : 0.4697, 과적합 Gap : 0.1700 , Test MSE : 0.002912 로 모델의 성능이 크게 상승하였다.



피쳐 엔지니어링 후, 피쳐 중요도를 시각화 해본 결과, 그리퍼의 바로 뒤에 위치한 관절인 **J4**와 **J5**가 각각 1위, 2위로 중요하고 파생 변수인 **is stationary**가 5위를 차지했다.

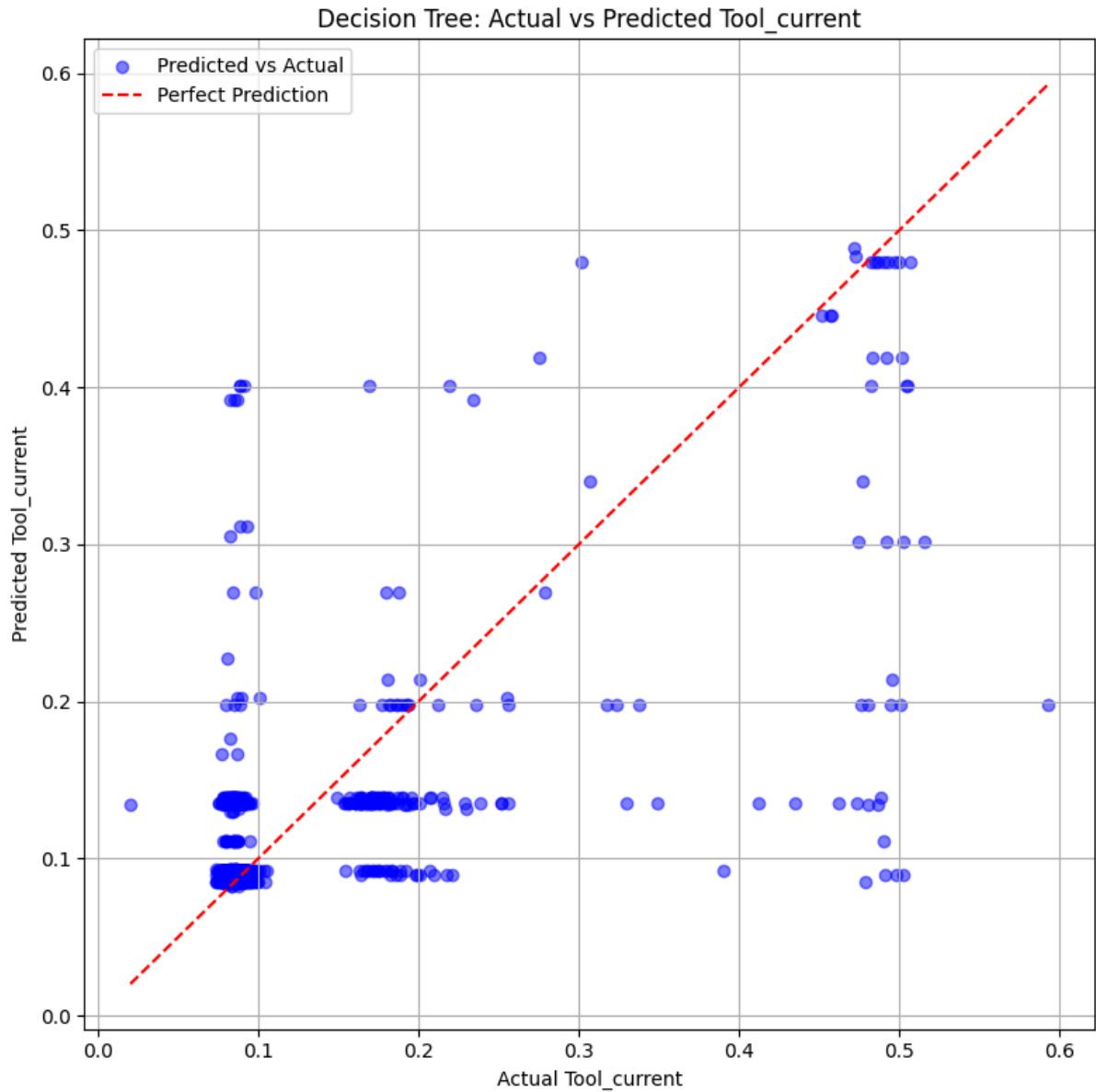
그리퍼 다음으로 하중을 많이 받는 부위가 **J4**와 **J5**이므로 이 변수들이 예측에 중요하다고 할 수 있는것은 타당하다고 할 수 있다. 파생 변수는 트리가 복잡하게 속도들을 조합하는 수고를 덜어주고 분기를 성공적으로 수행했음을 알려준다.

Decision Tree Structure Analysis (Top 3 Levels)



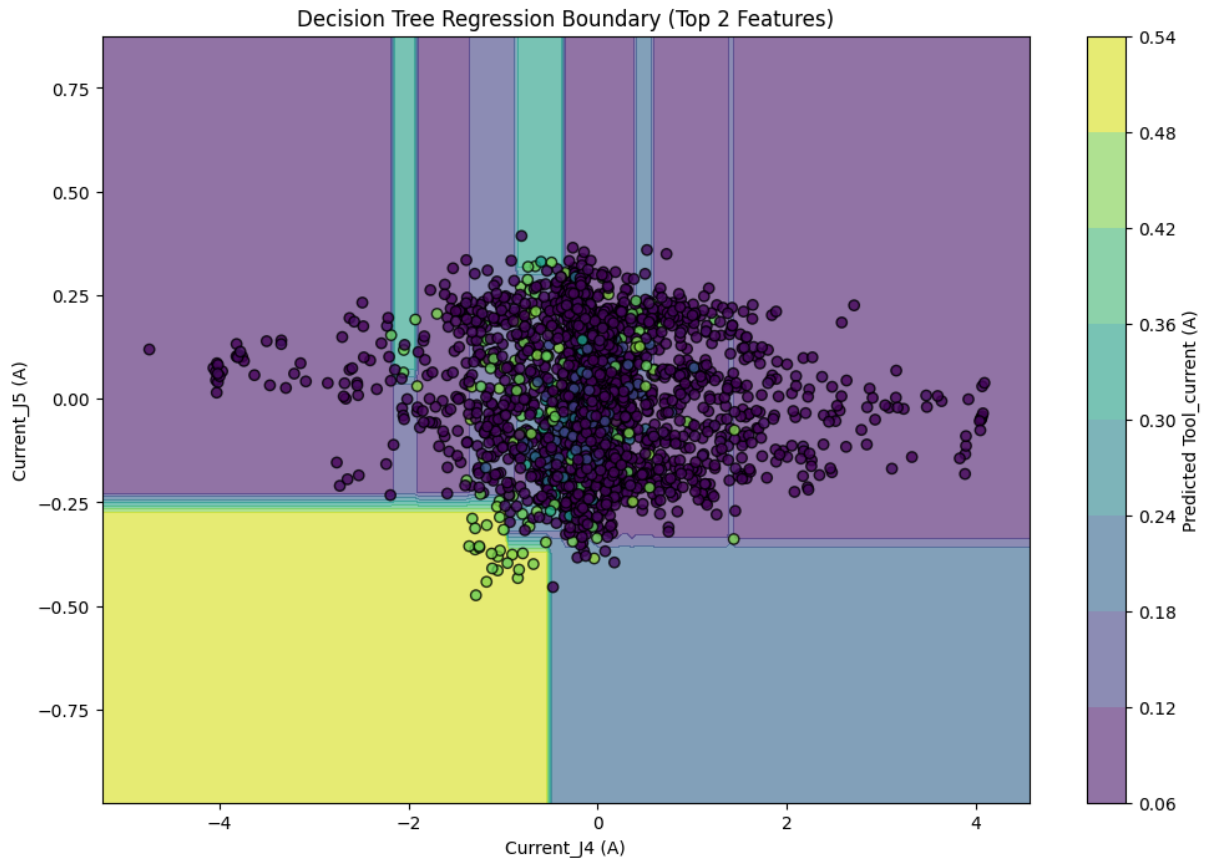
잘 보이지는 않지만, 학습한 의사결정나무를 시각화해보면, 최상단 루트 노드로 **Current J5** ≤ -0.335 를 통해 고부하 파지 상태를 최우선적으로 분단하고, 그 **True**는 전체 5,884개 데이터 중 단 30개만이 이 조건에 해당한다. 하지만 이 30개 데이터의 평균 예측값은 0.385로 매우 높으며, 그 아래 단계에서 최대 구동/파지 상태인 0.489 ~ 0.501로 수렴한다.

이어서 대다수 잔여 데이터에 대해 정지 상태 변수(**is_stationary** ≤ 0.5)를 핵심 분기 기준으로 채택하여 전체 제어 환경을 구동 상태와 정적 대기 상태로 이분화한다. 다음으로 동적 상태에서는 **Current J2**를 지표로 삼고, 정적 상태에서는 **Current J4**를 지표로 삼아 분기를 이어 나간다.



산점도 그래프를 보면 2.3절의 그리퍼 전류 분포와 같은 분포를 보여준다.

팔의 자세와 속도만으로는 그리퍼가 빈 손인지 물건을 쥐었는지 100% 확신할 수 없는 물리적 한계가 있고 실제 그리퍼는 이산적으로 동작한다고 할 수 있는데, 모델이 분기를 하다 헛갈리는 구간에서는 그 둘의 평균인 0.25A ~ 0.30A를 예측값으로 추정한다. 기계적으로 아예 존재하지도 않는 중간 전류값을 예측해버리니, R^2 성능이 오르는 데 근본적인 한계가 생긴다.



가장 중요한 관절 두개로 결정 경계를 시각화해 본 결과다.

중앙 상단의 얇은 세로 띠들은 데이터 몇 개에 반응하여 지나치게 좁은 영역을 분할한 결과일 가능성이 높다.

실제 데이터는 중앙에 매우 복잡하고 촘촘하게 모여 있다. 하지만 단일 트리 모델 특성상 경계면이 수직/수평으로만 단순하게 형성되어 복잡한 구역을 세밀하게 예측하지 못하고 있다. 이 때문에 전반적인 예측 성능이 떨어지는 것으로 보인다

이를 해결하기 위해, 수많은 트리를 생성하고 예측 결과를 평균 내어 직각의 결정 경계를 부드럽고 정교한 곡선 형태로 보완해 줄 수 있는 앙상블 기법인 랜덤 포레스트를 도입하고자 한다

4.2. Random Forest

decision tree의 단점을 보완하고 성능을 높이기 위해 앙상블 모델인 RF를 사용한다.

```

# 1. 교차 검증 (K-Fold) 설정
kf = KFold(n_splits=5, shuffle=True, random_state=42)

# 2. Random Forest Regressor 기본 모델 생성
# 병렬 처리(n_jobs=-1)를 추가하여 내부 트리 학습을 가속화합니다.
rf_reg = RandomForestRegressor(random_state=42)

# 3. 튜닝할 하이퍼파라미터 그리드 설정 (Decision Tree 탐색 결과 참고)
param_grid = {
    'n_estimators': [100, 200],          # 트리의 개수
    'max_depth': [10, 15, 20],           # 트리의 최대 깊이 (오버피팅 방지)
    'min_samples_leaf': [2, 5, 10],      # 리프 노드가 되기 위한 최소 샘플 수
    'min_samples_split': [5, 10, 20]     # 노드를 분할하기 위한 최소 샘플 수
}

# 4. GridSearchCV 실행 (R² 기준으로 최적 파라미터 탐색)
print("--- Random Forest 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 ---")
grid_search_rf = GridSearchCV(
    estimator=rf_reg,
    param_grid=param_grid,
    cv=kf,
    scoring='r2',
    n_jobs=-2, # CPU의 모든 코어를 사용하여 탐색 속도 대폭 상승
    verbose=1
)

grid_search_rf.fit(X_train, y_train)

# 5. 최적 모델 및 결과 확인
best_rf = grid_search_rf.best_estimator_

print("\n[GridSearchCV 결과]")
print("[최적의 하이퍼파라미터 설정]")
for param, value in grid_search_rf.best_params_.items():
    print(f"    • {param}: {value}")
print("")
print(f"최고 교차 검증 R² 점수: {grid_search_rf.best_score_:.4f}")

```

파생변수와 하이퍼파라미터 튜닝을 하기 전의 GridSearchCV를 진행해 보았다. 사용된 하이퍼 파라미터는 n estimators(트리의 개수), max depth(트리의 최대 깊이), min samples split (노드 분할을 위한 최소 샘플 수), min samples leaf (리프 노드가 되기 위한 최소 샘플 수)로 decision tree에서 트리의 개수만 추가되었다. 각각 N, D, L, S로 부르도록 하겠다.

```

--- Random Forest 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 ---
Fitting 5 folds for each of 54 candidates, totalling 270 fits

[GridSearchCV 결과]
[최적의 하이퍼파라미터 설정]
  • max_depth: 20
  • min_samples_leaf: 2
  • min_samples_split: 5
  • n_estimators: 200

최고 교차 검증 R2 점수: 0.5794

```

결과로, D가 20, L이 2, S가 5, 그리고 N이 200일때 최적의 성능을 보인다.

```

--- [모델 비교] 상위 10개 Random Forest 성능 종합 ---
(Random Forest는 무거워 재학습에 시간이 약간 소요될 수 있습니다)

```

	순위	CV R ²	Train R ²	Test R ²	과적합 Gap	Train MSE	Test MSE	파라미터 설정
0	1	0.5794	0.8801	0.5363	0.3439	0.000766	0.002547	N=200, D=20, L=2, S=5
1	2	0.5787	0.8733	0.5371	0.3362	0.000810	0.002542	N=200, D=15, L=2, S=5
2	3	0.5779	0.8801	0.5383	0.3418	0.000766	0.002535	N=100, D=20, L=2, S=5
3	4	0.5767	0.8731	0.5382	0.3349	0.000811	0.002536	N=100, D=15, L=2, S=5
4	5	0.5735	0.8380	0.5257	0.3123	0.001035	0.002605	N=200, D=20, L=2, S=10
5	6	0.5729	0.8324	0.5253	0.3072	0.001071	0.002607	N=200, D=15, L=2, S=10
6	7	0.5718	0.8389	0.5304	0.3086	0.001029	0.002579	N=100, D=20, L=2, S=10
7	8	0.5708	0.8334	0.5299	0.3035	0.001065	0.002582	N=100, D=15, L=2, S=10
8	9	0.5661	0.7850	0.5343	0.2508	0.001374	0.002558	N=100, D=20, L=5, S=5
9	9	0.5661	0.7850	0.5343	0.2508	0.001374	0.002558	N=100, D=20, L=5, S=10

```

=====
--- [상세 분석] 1위 최적 모델 최종 평가지표 ---
=====

[Train 데이터 평가]
R2 (결정계수) : 0.8801
MSE : 0.000766
RMSE : 0.027680
MAE : 0.010259

[Test 데이터 평가]
R2 (결정계수) : 0.5363
MSE : 0.002547
RMSE : 0.050465
MAE : 0.018846

```

결정나무에서 발생한 문제점이 그대로 발생했다. 과적합 gap이 0.3439로 굉장히 높아 과적합되었다고 할 수 있다.

```
# 3. 튜닝할 하이퍼파라미터 그리드 설정 (Decision Tree 탐색 결과 참고)
param_grid = {
    'n_estimators': [100, 200],           # 트리의 개수
    #'max_depth': [10, 15, 20],           # 트리의 최대 깊이 (오버피팅 방지)
    #'min_samples_leaf': [2, 5, 10],      # 리프 노드가 되기 위한 최소 샘플 수
    #'min_samples_split': [5, 10, 20]     # 노드를 분할하기 위한 최소 샘플 수
    'max_depth': [5, 6, 7, 8],
    'min_samples_leaf': [7, 10, 15, 20, 30], # 리프 노드 제한 범위를 더 넓혀봄
    'min_samples_split': [10, 20, 25, 30]
}
```

결정나무에서 사용했던 하이퍼파라미터를 사용한 결과

--- [모델 비교] 상위 10개 Random Forest 성능 종합 ---
(Random Forest는 무거워 재학습에 시간이 약간 소요될 수 있습니다)

	순위	CV R ²	Train R ²	Test R ²	과적합 Gap	Train MSE	Test MSE	파라미터 설정
0	1	0.5260	0.6674	0.4741	0.1933	0.002126	0.002888	N=200, D=8, L=7, S=10
1	2	0.5243	0.6681	0.4776	0.1905	0.002122	0.002869	N=100, D=8, L=7, S=10
2	3	0.5177	0.6525	0.4638	0.1887	0.002221	0.002945	N=200, D=8, L=7, S=20
3	4	0.5153	0.6529	0.4657	0.1872	0.002218	0.002934	N=100, D=8, L=7, S=20
4	5	0.5151	0.6413	0.4561	0.1852	0.002293	0.002987	N=200, D=8, L=10, S=20
5	5	0.5151	0.6413	0.4561	0.1852	0.002293	0.002987	N=200, D=8, L=10, S=10
6	7	0.5136	0.6388	0.4537	0.1851	0.002309	0.003000	N=200, D=8, L=7, S=25
7	8	0.5134	0.6417	0.4582	0.1835	0.002290	0.002975	N=100, D=8, L=10, S=10
8	8	0.5134	0.6417	0.4582	0.1835	0.002290	0.002975	N=100, D=8, L=10, S=20
9	10	0.5117	0.6382	0.4542	0.1840	0.002312	0.002998	N=100, D=8, L=7, S=25

--- [상세 분석] 1위 최적 모델 최종 평가지표 ---

[Train 데이터 평가]
R² (결정계수) : 0.6674
MSE : 0.002126
RMSE : 0.046110
MAE : 0.019946

[Test 데이터 평가]
R² (결정계수) : 0.4741
MSE : 0.002888
RMSE : 0.053743
MAE : 0.022805

2위 모델의 성능이 가장 뛰어나며 R2는 0.4776, 과적합 Gap은 0.1905를 기록했다.

여기서, N을 200과 100으로 한것의 차이가 매우 미미하므로, 학습 시간 단축과 효율성 증진을 위해 앞으로는 N을 100으로 고정하고, 전체 피쳐 중 몇 개만 랜덤으로 뽑아서 볼

것인지 결정하는 **max features** 파라미터(M)를 추가한다. 현재 10개 모델 모두 D를 최대치인 8으로 두고있어 D에 대한 값을 널널하게 바꾸고 다시 튜닝을 진행한다.

```
# 3. 튜닝할 하이퍼파라미터 그리드 설정 (Decision Tree 탐색 결과 참고)
param_grid = {
    'n_estimators': [100],          # 트리의 개수
    #'max_depth': [10, 15, 20],      # 트리의 최대 깊이 (오버피팅 방지)
    #'min_samples_leaf': [2, 5, 10], # 리프 노드가 되기 위한 최소 샘플 수
    #'min_samples_split': [5, 10, 20] # 노드를 분할하기 위한 최소 샘플 수

    'max_depth': [6, 8, 10],        # 기존 20 -> 12 이하로 대폭 축소
    'min_samples_leaf': [10, 20, 30], # 리프 노드 제한 범위를 더 넓혀봄
    'min_samples_split': [10, 20, 25],
    'max_features': ['sqrt', 0.5, 0.7, 1.0]
}
```

--- [모델 비교] 상위 10개 Random Forest 성능 종합 ---
(Random Forest는 무거워 재학습에 시간이 약간 소요될 수 있습니다)

	순위	CV R ²	Train R ²	Test R ²	과적합 Gap	Train MSE	Test MSE	파라미터 설정
0	1	0.5482	0.6765	0.5171	0.1594	0.002068	0.002652	D=10, L=10, S=10, M=0.5
1	1	0.5482	0.6765	0.5171	0.1594	0.002068	0.002652	D=10, L=10, S=20, M=0.5
2	3	0.5419	0.6817	0.5041	0.1776	0.002034	0.002723	D=10, L=10, S=10, M=0.7
3	3	0.5419	0.6817	0.5041	0.1776	0.002034	0.002723	D=10, L=10, S=20, M=0.7
4	5	0.5417	0.6688	0.5158	0.1530	0.002117	0.002659	D=10, L=10, S=25, M=0.5
5	6	0.5390	0.6664	0.5043	0.1621	0.002132	0.002722	D=10, L=10, S=25, M=0.7
6	7	0.5349	0.6574	0.5117	0.1457	0.002190	0.002682	D=10, L=10, S=10, M=sqrt
7	7	0.5349	0.6574	0.5117	0.1457	0.002190	0.002682	D=10, L=10, S=20, M=sqrt
8	9	0.5298	0.6484	0.5048	0.1436	0.002247	0.002719	D=10, L=10, S=25, M=sqrt
9	10	0.5298	0.6785	0.4841	0.1944	0.002055	0.002833	D=10, L=10, S=10, M=1.0

--- [상세 분석] 1위 최적 모델 최종 평가지표 ---

[Train 데이터 평가]
R² (결정계수) : 0.6765
MSE : 0.002068
RMSE : 0.045470
MAE : 0.018689

[Test 데이터 평가]
R² (결정계수) : 0.5171
MSE : 0.002652
RMSE : 0.051496
MAE : 0.020964

과적합 gap은 줄어 들고 test R2 값은 늘어나는 결과를 볼 수 있었다.

여러번 하이퍼파라미터 튜닝을 시도한 결과, 아래와 같은 결과들이 나왔다.

--- [모델 비교] 상위 10개 Random Forest 성능 종합 ---
(Random Forest는 무거워 재학습에 시간이 약간 소요될 수 있습니다)

	순위	CV R ²	Train R ²	Test R ²	과적합 Gap	Train MSE	Test MSE	파라미터 설정
0	1	0.5904	0.8285	0.5518	0.2767	0.001096	0.002461	D=16, L=3, S=8, M=0.6
1	2	0.5889	0.8171	0.5562	0.2608	0.001169	0.002437	D=16, L=3, S=8, M=0.4
2	3	0.5879	0.8188	0.5576	0.2612	0.001158	0.002429	D=14, L=3, S=8, M=0.5
3	4	0.5878	0.8262	0.5486	0.2777	0.001111	0.002479	D=16, L=3, S=8, M=0.5
4	5	0.5876	0.8260	0.5556	0.2704	0.001112	0.002441	D=14, L=3, S=8, M=0.6
5	6	0.5874	0.8074	0.5604	0.2471	0.001231	0.002414	D=14, L=3, S=10, M=0.5
6	7	0.5874	0.8100	0.5535	0.2566	0.001214	0.002452	D=14, L=3, S=8, M=0.4
7	8	0.5867	0.8078	0.5456	0.2622	0.001229	0.002495	D=14, L=3, S=10, M=0.6
8	9	0.5856	0.8079	0.5477	0.2602	0.001228	0.002484	D=12, L=3, S=8, M=0.5
9	10	0.5856	0.7939	0.5492	0.2447	0.001318	0.002476	D=14, L=3, S=10, M=0.4

이 시도에서는 6위 모델이 다른 모델들에 비해 test R2가 0.5604 까지 올라왔으나 과적합 gap은 늘어났다.

	순위	CV R ²	Train R ²	Test R ²	과적합 Gap	Train MSE	Test MSE	파라미터 설정
0	1	0.5553	0.6969	0.5209	0.1761	0.001937	0.002631	D=11, L=9, S=10, M=0.5
1	2	0.5531	0.6923	0.5233	0.1690	0.001967	0.002618	D=10, L=9, S=10, M=0.6
2	3	0.5530	0.6887	0.5267	0.1620	0.001989	0.002599	D=11, L=9, S=10, M=0.4
3	4	0.5526	0.6898	0.5186	0.1711	0.001983	0.002644	D=10, L=9, S=10, M=0.5
4	5	0.5524	0.7025	0.5234	0.1791	0.001902	0.002617	D=11, L=9, S=10, M=0.6
5	6	0.5506	0.6989	0.5111	0.1878	0.001924	0.002685	D=11, L=9, S=10, M=0.7
6	7	0.5486	0.6861	0.5140	0.1721	0.002006	0.002669	D=11, L=10, S=10, M=0.5
7	8	0.5482	0.6768	0.5182	0.1587	0.002066	0.002646	D=10, L=9, S=10, M=0.4
8	9	0.5482	0.6765	0.5171	0.1594	0.002068	0.002652	D=10, L=10, S=10, M=0.5
9	10	0.5471	0.6922	0.5207	0.1715	0.001967	0.002632	D=11, L=10, S=10, M=0.6

이 시도에서는 3위 모델이 R2가 0.5267, gap이 0.1620로 나왔다.

파생 변수 없이 진행한 튜닝의 결과를 정리하자면,

가장 Test R2가 높은 결과는

D=14,L=3,S=8,M=0.5인 모델이며 R2 = 0.5604, MSE = 0.002414, Gap = 0.2471

가장 과적합되지 않은 결과는

D=10,L=10,S=10,M=0.5인 모델이며 R2 = 0.5171, MSE = 0.002652, Gap = 0.1594

가장 균형있는 결과는

D=11,L=9,S=10,M=0.4인 모델이며 R2 = 0.5267, MSE = 0.002599, Gap = 0.1690

으로 결과가 나왔다.

파생변수인 정지 여부를 판별하는 is stationary를 넣고 다시 학습을 진행한다.

--- [모델 비교] 상위 10개 Random Forest 성능 종합 ---

(Random Forest는 무거워 재학습에 시간이 약간 소요될 수 있습니다)

	순위	CV R ²	Train R ²	Test R ²	과적합 Gap	Train MSE	Test MSE	파라미터 설정
0	1	0.5488	0.6788	0.5315	0.1473	0.002053	0.002573	D=12, L=10, S=10, M=0.4
1	2	0.5483	0.6826	0.5369	0.1457	0.002028	0.002543	D=14, L=10, S=10, M=0.4
2	3	0.5468	0.6863	0.5387	0.1476	0.002005	0.002534	D=14, L=10, S=10, M=0.5
3	4	0.5464	0.6836	0.5349	0.1487	0.002022	0.002554	D=12, L=10, S=10, M=0.5
4	5	0.5453	0.6567	0.5165	0.1402	0.002194	0.002655	D=12, L=12, S=10, M=0.4
5	6	0.5449	0.6676	0.5264	0.1412	0.002124	0.002601	D=10, L=10, S=10, M=0.4
6	7	0.5435	0.6609	0.5250	0.1359	0.002167	0.002608	D=14, L=12, S=10, M=0.4
7	8	0.5418	0.6715	0.5202	0.1513	0.002099	0.002635	D=10, L=10, S=10, M=0.5
8	9	0.5384	0.6623	0.5199	0.1424	0.002159	0.002637	D=12, L=12, S=10, M=0.5
9	10	0.5374	0.6508	0.5178	0.1330	0.002232	0.002648	D=10, L=12, S=10, M=0.4

파생변수를 넣자 모델들의 R2가 상승하고 gap이 줄어든 결과들을 확인할 수 있다.

눈에 띄는 모델은 3위 모델로

D=14, L=10, S=10, M=0.5 인 경우 R2 = 0.5387, MSE = 0.002534, Gap = 0.1476 으로

파생변수를 넣기 전보다 성능이 올랐다.

--- [모델 비교] 상위 10개 Random Forest 성능 종합 ---

(Random Forest는 무거워 재학습에 시간이 약간 소요될 수 있습니다)

	순위	CV R ²	Train R ²	Test R ²	과적합 Gap	Train MSE	Test MSE	파라미터 설정
0	1	0.5560	0.7110	0.5425	0.1686	0.001847	0.002513	D=14, L=8, S=15, M=0.4
1	1	0.5560	0.7110	0.5425	0.1686	0.001847	0.002513	D=14, L=8, S=8, M=0.4
2	1	0.5560	0.7110	0.5425	0.1686	0.001847	0.002513	D=14, L=8, S=10, M=0.4
3	4	0.5557	0.7065	0.5430	0.1635	0.001876	0.002510	D=12, L=8, S=8, M=0.5
4	4	0.5557	0.7065	0.5430	0.1635	0.001876	0.002510	D=12, L=8, S=15, M=0.5
5	4	0.5557	0.7065	0.5430	0.1635	0.001876	0.002510	D=12, L=8, S=10, M=0.5
6	7	0.5555	0.7134	0.5433	0.1700	0.001832	0.002508	D=14, L=8, S=8, M=0.5
7	7	0.5555	0.7134	0.5433	0.1700	0.001832	0.002508	D=14, L=8, S=10, M=0.5
8	7	0.5555	0.7134	0.5433	0.1700	0.001832	0.002508	D=14, L=8, S=15, M=0.5
9	10	0.5555	0.7068	0.5455	0.1613	0.001874	0.002496	D=12, L=8, S=15, M=0.4

--- [모델 비교] 상위 10개 Random Forest 성능 종합 ---
(Random Forest는 무거워 재학습에 시간이 약간 소요될 수 있습니다)

	순위	CV R ²	Train R ²	Test R ²	과적합 Gap	Train MSE	Test MSE	파라미터 설정
0	1	0.5832	0.8164	0.5766	0.2398	0.001174	0.002325	D=16, L=3, S=8, M=0.4
1	2	0.5828	0.7952	0.5685	0.2267	0.001309	0.002370	D=14, L=3, S=10, M=0.4
2	3	0.5825	0.8176	0.5735	0.2442	0.001166	0.002342	D=14, L=3, S=8, M=0.5
3	4	0.5825	0.8155	0.5735	0.2419	0.001180	0.002342	D=14, L=3, S=8, M=0.6
4	5	0.5821	0.8016	0.5668	0.2348	0.001268	0.002379	D=16, L=3, S=10, M=0.4
5	6	0.5818	0.8098	0.5705	0.2392	0.001216	0.002359	D=14, L=3, S=8, M=0.4
6	7	0.5812	0.7992	0.5660	0.2332	0.001283	0.002383	D=12, L=3, S=8, M=0.4
7	8	0.5804	0.8211	0.5848	0.2363	0.001143	0.002280	D=16, L=3, S=8, M=0.5
8	9	0.5802	0.7997	0.5765	0.2232	0.001280	0.002326	D=14, L=3, S=10, M=0.5
9	10	0.5796	0.7999	0.5773	0.2226	0.001279	0.002322	D=12, L=3, S=8, M=0.5

=====

--- [상세 분석] 1위 최적 모델 최종 평가지표 ---

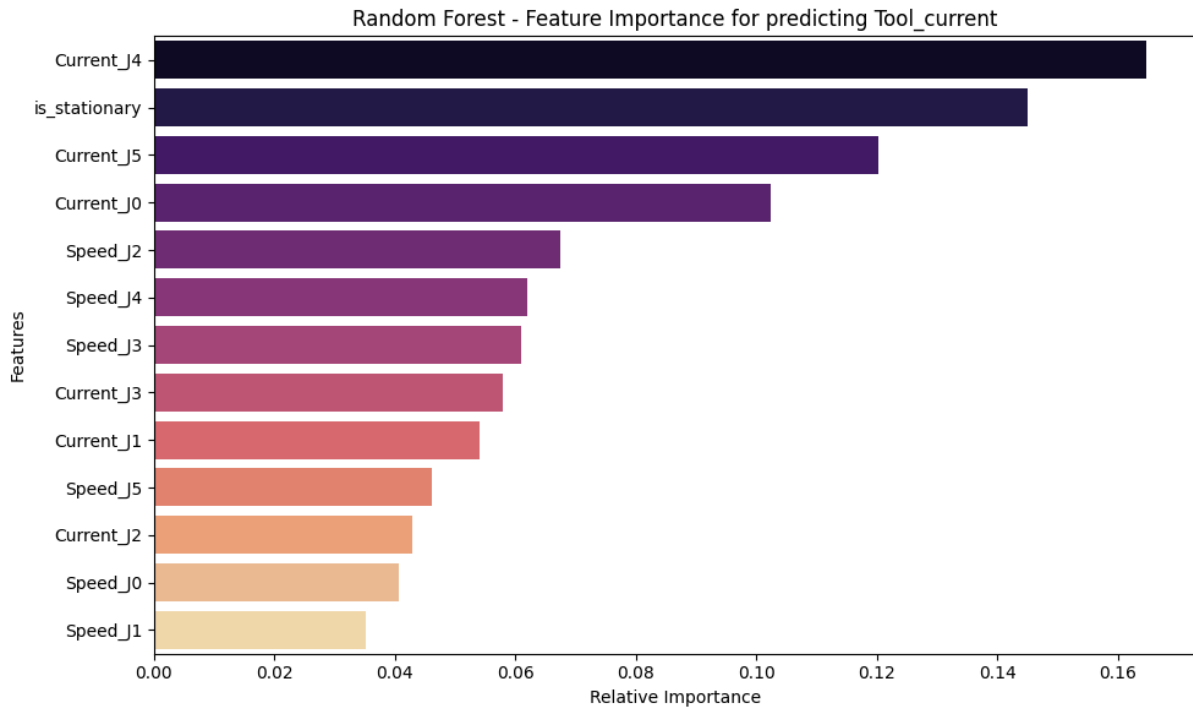
=====

[Train 데이터 평가]
R² (결정계수) : 0.8164
MSE : 0.001174
RMSE : 0.034257
MAE : 0.013189

[Test 데이터 평가]
R² (결정계수) : 0.5766
MSE : 0.002325
RMSE : 0.048222
MAE : 0.018645

튜닝 결과, 가장 높은 R2 값인 0.5848이 등장했다. 해당 모델의 MSE는 0.002280으로 지금까지 중 가장 작지만, gap이 0.2363으로 다른 모델들과 같이 과적합되어있다.

하이퍼파라미터 튜닝을 통해 모델의 성향을 결정짓는 가장 중요한 변수는 바로 min samples leaf 라는 점을 발견했다. 트리의 깊이보다도 L값이 모델의 운명을 좌우하고 있다. L을 3으로 데이터가 3개만 남아도 분할을 허용하여 트리가 데이터를 아주 세밀하게 쪼개도록 두자 Test R2가 0.5848로 가장 높았다. L을 9 ~ 12으로 넉넉하게 두면 과적합 Gap이 0.14~0.15으로 방어되고 Test R2가 0.52~0.53 에서 머문다.



랜덤포레스트의 피쳐 중요도 그래프를 보니, **Decision Tree**의 중요도와는 다르게 **J4, is stationary** 변수가 1위 2위를 차지하고, **J5**가 3위를 차지했다. 또한 단일 트리에서는 눈에 띄지 않던 **J0**가 그 다음으로 중요한 피쳐로 떠올랐는데 이는 앙상블 기법을 통해 다각도로 데이터를 분석하자 **J0**에 걸리는 부하가 그리퍼의 상태를 보여주는 피쳐라는것이 드러난 결과로 해석할 수 있겠다.

단일 트리의 피쳐 중요도 그래프에서는 그 밖의 피쳐들의 중요도가 0에 가까웠는데 **RF**에서는 여러 트리가 각자 다른 변수 조합을 바탕으로 학습하고 이를 평균냈기에 특정 1~2개 변수에만 의존하여 과적합되던 단일 트리의 단점이 극복되고, 모든 센서 데이터의 미세한 정보들을 골고루 끌어모아 예측의 안정성을 높인 앙상블의 효과때문에 막대가 늘어났다.

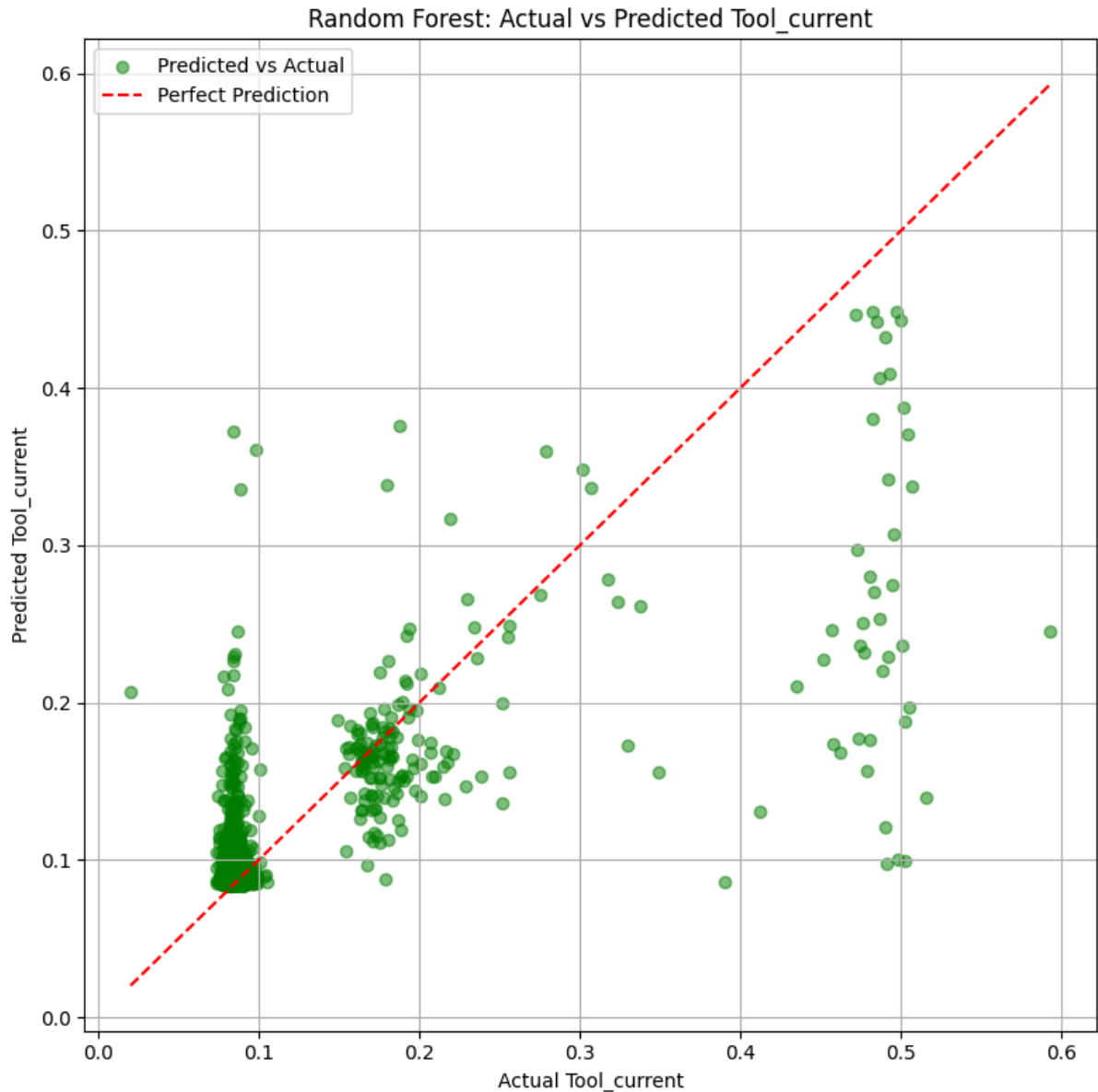
그럼, 가장 높게나왔던 **J4,J5,is stationary**의 5개 피쳐만으로 랜덤포레스트를 해보면

--- [모델 비교] 상위 10개 Random Forest 성능 종합 ---
 (Random Forest는 무거워 재학습에 시간이 약간 소요될 수 있습니다)

	순위	CV R ²	Train R ²	Test R ²	과적합 Gap	Train MSE	Test MSE	파라미터 설정
0	1	0.4679	0.5800	0.4241	0.1559	0.002684	0.003163	D=14, L=10, S=10, M=0.5
1	1	0.4679	0.5800	0.4241	0.1559	0.002684	0.003163	D=14, L=10, S=10, M=0.4
2	3	0.4670	0.5777	0.4180	0.1597	0.002699	0.003196	D=12, L=10, S=10, M=0.5
3	3	0.4670	0.5777	0.4180	0.1597	0.002699	0.003196	D=12, L=10, S=10, M=0.4
4	5	0.4664	0.5713	0.4235	0.1478	0.002740	0.003166	D=10, L=10, S=10, M=0.4
5	5	0.4664	0.5713	0.4235	0.1478	0.002740	0.003166	D=10, L=10, S=10, M=0.5
6	7	0.4662	0.5656	0.4157	0.1499	0.002776	0.003209	D=14, L=12, S=10, M=0.4
7	7	0.4662	0.5656	0.4157	0.1499	0.002776	0.003209	D=14, L=12, S=10, M=0.5
8	9	0.4654	0.5627	0.4148	0.1479	0.002795	0.003214	D=12, L=12, S=10, M=0.4
9	9	0.4654	0.5627	0.4148	0.1479	0.002795	0.003214	D=12, L=12, S=10, M=0.5

가장 좋은 모델인 1위 모델만 보더라도 R²가 많이 떨어졌고 gap은 원래와 비슷하지만 지금은 Train 성능과 Test 성능이 동반 하락하면서 gap이 0.14 ~ 0.15으로 유지된 형태다. 이는 모델이 일반화되었다기보다는, 학습할 재료가 너무 부족해져서 데이터의 패턴 자체를 충분히 설명하지 못하는 과소적합에 가까워진 상태다.

가장 중요한 5개만 남겼음에도 Test R²가 0.54 에서 0.42 수준으로 떨어졌다는 것은, 제외된 8개의 피쳐 중에 핵심적인 정보가 섞여 있었다는 뜻이다. Random Forest의 피쳐 중요도는 개별 변수의 불순도 감소량을 기준으로 측정되기 때문에, 하위권으로 밀려났던 8개의 피쳐들이 단독으로는 눈에 띄지 않더라도, 상위 피쳐들과 상호작용하면서 미세한 오차와 변수들을 잡아주는 역할을 하고 있었던 것으로 보인다.



이전 **Decision Tree**에서는 예측값이 특정 수평선에 종종 갇혀있는 부자연스러운 계단화 현상이 심했지만, 앙상블 기법을 통해 수많은 트리의 결과를 평균 내었기 때문에, Y축 방향으로 점들이 부드럽게 퍼져 있다. 가로로 줄이 그어져 있던 **DT**의 부자연스러움이 해소되었고, 이 덕분에 전체적인 **R2** 성능이 **DT**보다 상승한 것이다.

성능은 올랐지만, 왜 과적합 **Gap**이 여전히 존재하고 완벽한 예측이 불가능한지 그 이유를 잘 보여준다. **0.5A**인 고부하 상태의 점들을 보면, 빨간색 점선에 닿아있는 점은 거의 없고, 대다수의 초록 점들이 **0.1**에서 **0.45** 사이에 밀으로 흘러내리듯 분포하고 있다.

학습 데이터의 **80%**가 **0.08A**이기 때문에, **RF** 내부의 수많은 트리들은 편향 학습이 되어있다.

진짜 고부하 상태의 센서 데이터가 들어와도, 앙상블 내부의 일부 트리들이 낮은 값을 예측하여 전체 평균을 밑으로 끌어내려 버리기 때문이다.

또한, 0.25A ~ 0.45A 사이에는 실제 데이터가 텅 비어 있다. 즉, 존재하지 않는 물리적 상태이다. 하지만 예측값을 보면 이 빈 공간에 많은 초록 점들이 예측되어 있다. 이유는 간단하게 트리가 0.08A 상태와 0.5A 상태를 헛갈려 할 때, 두 값을 더해 평균을 내버리기 때문이다. 기계는 0.08 아니면 0.5로 확고하게 움직이는데, 회귀 모델인 RF는 두 값을 평균내버리는 물리적 오류를 일으키기에 여전히 과적합이 높다고 할 수 있다.

5.K-Nearest Neighbor

선형회귀, 결정 트리, 랜덤 포레스트가 모델의 매개변수나 분기 규칙을 생성하는 것과 달리, KNN은 별도의 학습 과정 없이 주변 이웃 데이터 K개 만을 참조하여 결과를 내기때문에 이전의 결과들과 비교할 가치가 있다고 생각한다.

```
# 2. 결측치 없는 클린 데이터 분리
df_clean = df.dropna().copy()

# -----
# [유지] 2.5 도메인 지식 기반 파생 변수 (Feature Engineering)
# -----
speed_cols = ['Speed_J0', 'Speed_J1', 'Speed_J2', 'Speed_J3', 'Speed_J4', 'Speed_J5']
df_clean['is_stationary'] = (df_clean[speed_cols].abs().sum(axis=1) < 0.05).astype(int)

# 3. 입력 피쳐(x) 및 타겟(y) 설정
input_cols = [
    'Current_J0', 'Current_J1', 'Current_J2', 'Current_J3', 'Current_J4', 'Current_J5',
    'Speed_J0', 'Speed_J1', 'Speed_J2', 'Speed_J3', 'Speed_J4', 'Speed_J5',
    'is_stationary'
]
x = df_clean[input_cols]
y = df_clean['Tool_current']

print(f"데이터 크기: {df_clean.shape}")
print(f"피쳐 개수: {len(input_cols)}개, 타겟: Tool_current")
```

✓ 0.0s

데이터 크기: (7355, 25)

피쳐 개수: 13개, 타겟: Tool_current

우선 처음부터 파생 변수인 is stationary와 다른 12개의 피쳐를 전부 사용해 학습을 진행한다.

```

# 1. 80:20으로 Train / Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. 데이터 스케일링 [중요]
# 거리 기반 알고리즘인 KNN은 변수의 단위가 매우 중요하므로 StandardScaler를 적용합니다.
scaler = StandardScaler()

# Train 데이터로 기준을 학습(fit)하고 변환(transform)
X_train_scaled = scaler.fit_transform(X_train)

# Test 데이터는 Train의 기준을 그대로 사용하여 변환(transform)만 수행
X_test_scaled = scaler.transform(X_test)

# 결과를 다시 DataFrame으로 감싸기 (가독성을 위함)
X_train_scaled = pd.DataFrame(X_train_scaled, columns=input_cols)
X_test_scaled = pd.DataFrame(X_test_scaled, columns=input_cols)

print(f"스케일링 전 Train 1번 데이터: \n{X_train.iloc[0].values[:4]} ...")
print(f"스케일링 후 Train 1번 데이터: \n{X_train_scaled.iloc[0].values[:4]} ...")

```

✓ 0.0s

스케일링 전 Train 1번 데이터:
 [-0.12960072 -4.23103476 -1.79280257 -1.56214726] ...

스케일링 후 Train 1번 데이터:
 [-0.09911143 -2.3880921 -0.95897061 -1.83983177] ...

Knn은 거리기반 알고리즘이기에 변수의 단위가 다르면 모델이 피쳐들의 중요도를 다르게 계산하기에 평균을 0, 표준편차를 1로 만드는 스탠다드 스케일링을 사용해 진행한다.

```

# 1. 교차 검증 (K-Fold) 설정
kf = KFold(n_splits=5, shuffle=True, random_state=42)

# 2. KNN Regressor 기본 모델 생성
knn_reg = KNeighborsRegressor(n_jobs=-1)

# 3. 튜닝할 하이퍼파라미터 그리드 설정
param_grid = {
    'n_neighbors': [3, 4, 5, 6, 7, 8, 9, 10, 15, 20], # 이웃의 수 (과적합~과소적합 탐색)
    'weights': ['uniform', 'distance'], # uniform: 동일 가중치, distance: 가까울수록 큰 가중치
    'p': [1, 2] # 1: 맨해튼 거리, 2: 유클리디안 거리
}

# 4. GridSearchCV 실행
# KNN은 학습 자체는 즉시 끝나지만, 평가 시 거리를 일일이 계산하므로 시간이 다소 걸릴 수 있습니다.
print("--- KNN 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 ---")
grid_search_knn = GridSearchCV(
    estimator=knn_reg,
    param_grid=param_grid,
    cv=kf,
    scoring='r2',
    n_jobs=-1,
    verbose=1
)

# [주의] 반드시 스케일링된 데이터(X_train_scaled)를 넣어야 합니다.
grid_search_knn.fit(X_train_scaled, y_train)

# 5. 최적 모델 및 결과 확인
best_knn = grid_search_knn.best_estimator_

print("\n[GridSearchCV 결과]")
print("[최적의 하이퍼파라미터 설정]")
for param, value in grid_search_knn.best_params_.items():
    print(f"    • {param}: {value}")
print("")
print(f"최고 교차 검증 R² 점수: {grid_search_knn.best_score_:.4f}")

```

KNN 모델에 대한 하이퍼파라미터 튜닝을 진행한다.

`n_neighbors` 는 거리를 측정할 이웃의 수 `K`개를 나타낸다.

`weights` 는 예측할 때 이웃들의 투표 결과나 값에 가중치를 부여하는 방식을 결정한다.

`uniform`은 거리에 상관없이 모든 이웃이 동일한 가중치를 가진다. `distance`는 가까운 이웃일수록 더 큰 가중치를 가지고 거리에 반비례하여 영향을 준다.

`p`는 거리를 측정하는 방식을 결정하는 파라미터인데 `1`은 맨해튼, `2`는 유클리디안 거리로 측정한다.

```

--- KNN 하이퍼파라미터 탐색 및 5-Fold 교차 검증 진행 중 ---
Fitting 5 folds for each of 40 candidates, totalling 200 fits

[GridSearchCV 결과]
[최적의 하이퍼파라미터 설정]
    • n_neighbors: 8
    • p: 1
    • weights: distance

최고 교차 검증 R² 점수: 0.5341

```

--- [모델 비교] 상위 10개 KNN 성능 종합 ---

	순위	CV R ²	Train R ²	Test R ²	과적합 Gap	Train MSE	Test MSE	파라미터 설정
0	1	0.5341	1.0000	0.5025	0.4975	0.000000	0.002732	K=8, W=dist, P=1
1	2	0.5336	1.0000	0.5013	0.4987	0.000000	0.002739	K=9, W=dist, P=1
2	3	0.5324	1.0000	0.5031	0.4969	0.000000	0.002729	K=7, W=dist, P=1
3	4	0.5309	1.0000	0.4981	0.5019	0.000000	0.002756	K=10, W=dist, P=1
4	5	0.5302	1.0000	0.4988	0.5012	0.000000	0.002753	K=6, W=dist, P=1
5	6	0.5265	1.0000	0.4889	0.5111	0.000000	0.002807	K=5, W=dist, P=1
6	7	0.5245	0.6488	0.5022	0.1466	0.002245	0.002734	K=8, W=uni, P=1
7	8	0.5233	0.6645	0.5045	0.1600	0.002145	0.002721	K=7, W=uni, P=1
8	9	0.5229	1.0000	0.5219	0.4781	0.000000	0.002626	K=9, W=dist, P=2
9	10	0.5228	0.6397	0.4984	0.1412	0.002303	0.002754	K=9, W=uni, P=1

1위 모델을 포함하여 **distance**를 기반으로 한 모델은 전부 **Train R2**가 1.0이라는 값이 나왔다. 이것은 과적합된것이 아니라 **distance** 가중치는 예측하려는 데이터와 이웃 데이터 간의 거리가 가까울수록 무한대에 가까운 가중치를 준다.

Train 데이터 자체를 예측할 때는 자기 자신과의 거리가 0 이 되므로, 다른 이웃을 무시하고 무조건 자기 자신의 정답을 출력하게 되어 1.0으로 결과가 나와 과적합 **Gap**이 크게 벌어졌다.

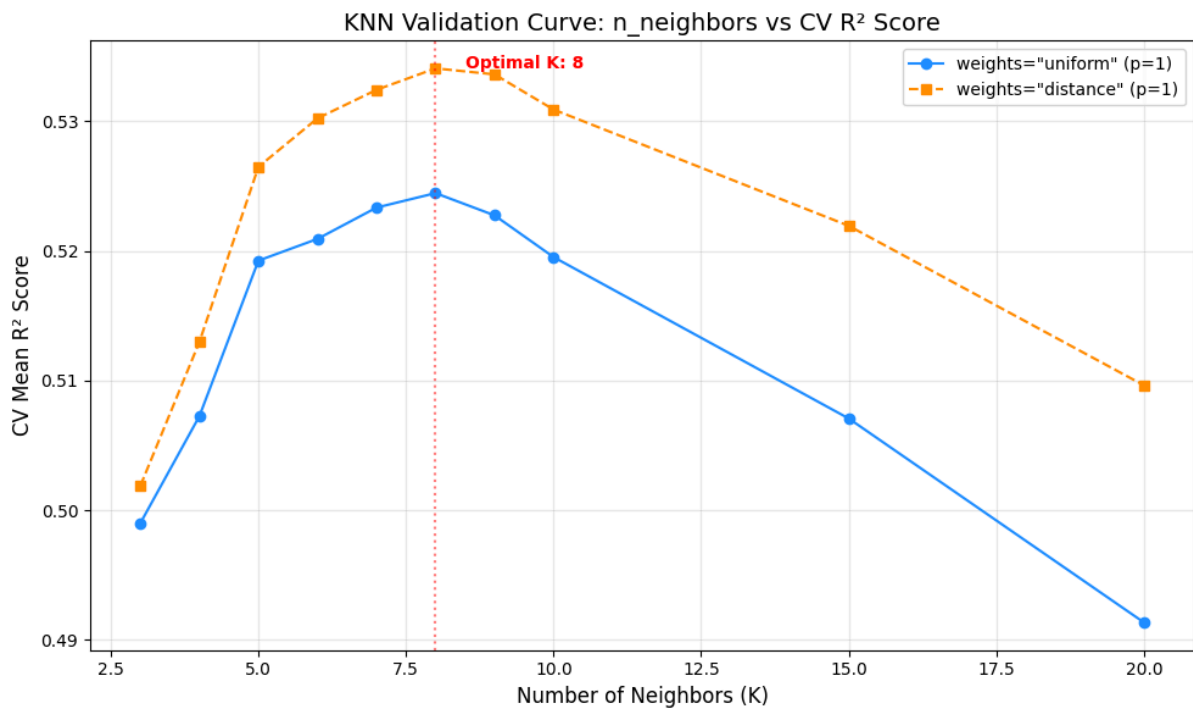
7위 모델을 보면 **Train R2**가 0.6645, **Test R2**가 0.5022로 정상적이다. 즉, Knn 모델 자체는 정상적으로 학습되었으며 따라서 **distance**를 쓴 KNN 모델을 평가할 때는 **Train R2**와 **Gap**은 무시하시고, 오직 **CV R2**와 **Test R2**만 보고 평가하시는 것이 좋겠다.

표의 7위, 8위를 보면 **uniform**을 가중치로 사용했다. 이 경우 **Train R2**는 0.64 ~ 0.66으로 똑 떨어지고, 과적합 **Gap**은 0.14~0.16으로 **Random Forest** 때와 비슷해진다.

이전 **Random Forest** 모델이 완전히 동일한 로봇 자세에서 0.08A와 0.49A가 동시에 존재하여, **RF**가 그 둘을 평균 내버려서 빈 공간을 예측하는 오류를 범했는데 KNN의 **uniform** 역시 똑같은 오류를 낸다. 이웃 K개 중에 0.08A와 0.49A가 섞여 있으면 그냥 산술 평균을 내버리기 때문에 성능이 오르지 못하는 것이다.

반면 **distance**는 미세하게라도 더 가까운 점의 전류값을 강하게 끌어당기기 때문에, 조금 더 성능이 올랐다고 추측할 수 있다.

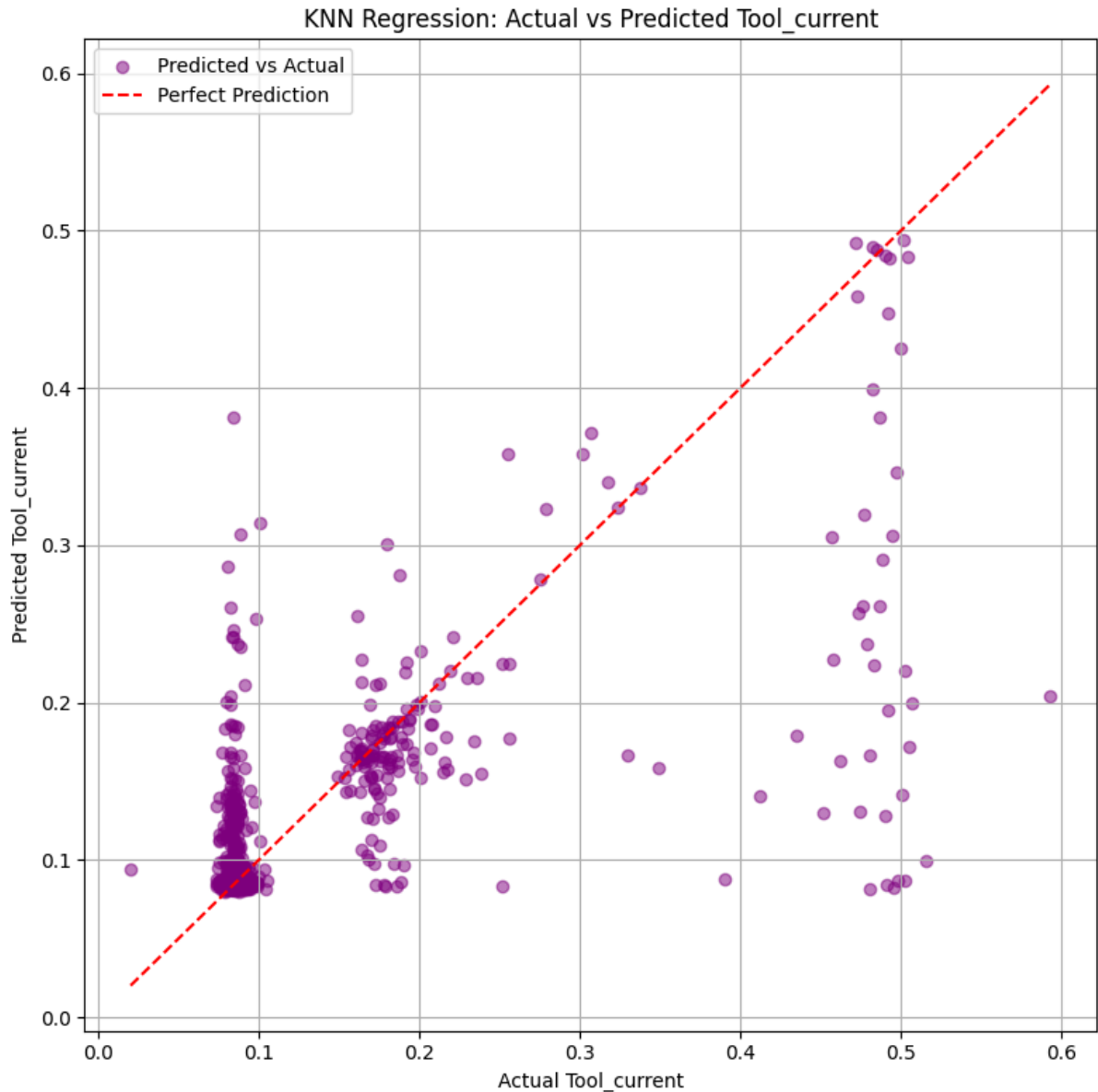
튜닝 결과 가장 성능이 좋은 모델은 9위 모델로 Test R2가 0.5219로 전체 표에서 가장 높고 MSE 역시 0.002626으로 가장 낮다. CV R2와 Test R2를 비교해보면 점수의 차이가 0.001에 불과하다.



검증 곡선 그래프를 보면 **distance**가 **uniform**에 비해 항상 위에 있다. **regression**의 **uniform** 방식은 이웃들의 값을 산술 평균내어 값을 예측하기에 실제 정답값에 없는 값을 예측하기에 오류가 커진다. **distance** 방식은 더 가까운 데이터에 강한 가중치를 부여함으로써 중첩된 공간속에서도 정답에 더 가깝게 달라붙는 능력이 뛰어나를 증명한다.

최적의 K 값은 8로 나왔다. K가 8보다 작다면 너무 민감하게 반응하여 과적합이 발생해 성능이 낮아지고 K가 8보다 커질수록 예측 성능이 감소하는데 이유를 추론할 수 있다. **tool current**의 대다수는 0.08A에 밀집되어있어 모델이 참조하는 이웃의 반경을 너무 넓히게 되면, 실제로는 0.49A의 데이터를 예측해야할 때, 수많은 0.08A 데이터들이 이웃으로 편입되어 과소적합이 발생하여 성능이 하락한다.

따라서 **Knn**에서는 노이즈를 상쇄할만큼 많은 데이터를 참조하면서도 다른 상태의 데이터까지 섞이지 않는 가장 이상적인 K의 크기가 8개 내외임을 시사한다.



산점도 그래프를 보면 **RF**와 크게 다르지 않다. 이는 거리 기반 알고리즘인 **KNN**이 이웃을 탐색할 때, 고부하 상태의 데이터와 대기 상태의 데이터가 입력 공간 상에서 완전히 중첩되어 구별되지 않음을 의미한다. 결과적으로 압도적 다수를 차지하는 대기 상태 데이터가 인접 이웃으로 대거 편입되어 예측값을 끌어내린 것이다.

6.결론

6.1. 프로젝트 요약 및 모델별 성능 종합

이로써 **Regression Task**를 마쳤다.

주제는 **J0 ~ J5**까지 6개의 관절의 **current**, **speed**로 **tool current**의 값을 예측하는 것이었다.

모델은 **Linear Regression**, **Decision Tree**, **Random Forest**, **K-Nearest Neighbor** 4가지를 사용했다.

Linear Regression에서 EDA 단계에서 확인된 극단적인 비선형성과 관절간의 다중공산성으로 인해 Test R2가 0.23 수준에 그쳤다.

Decision Tree에서 비선형 경계를 학습하기 위해 트리 모델을 도입하였고, 파생 변수 is stationary를 추가하여 Test R2를 0.47까지 올렸다.

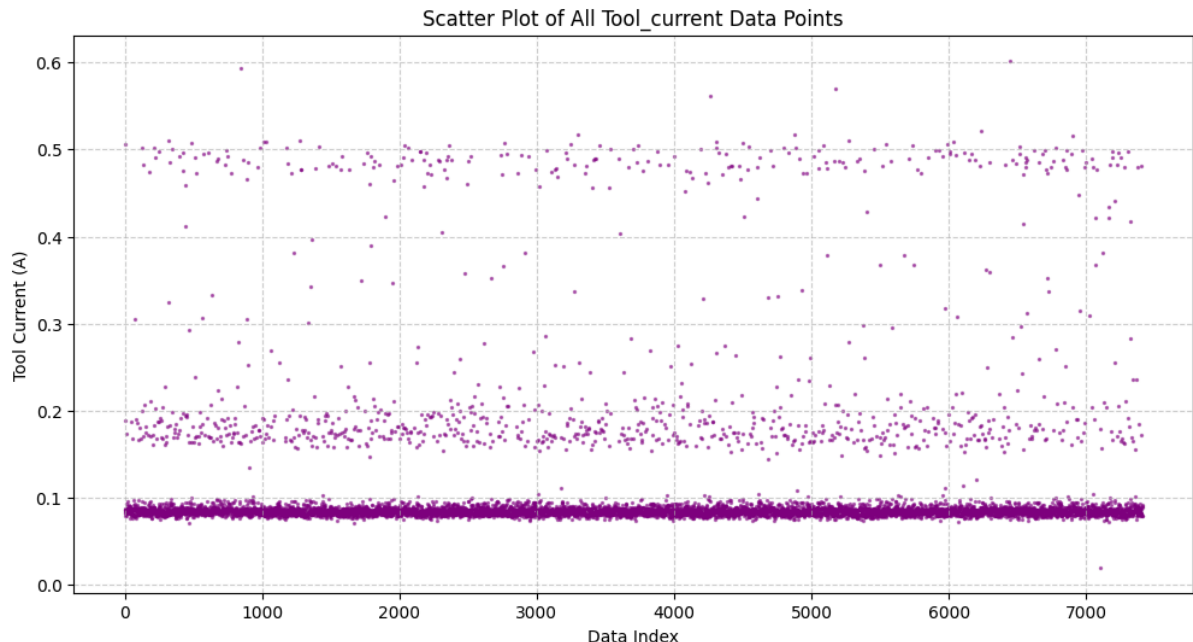
Random Forest에서 앙상블 기법을 통해 단일 트리의 단점을 보완하며 최고 성능인 Test R2가 0.5848을 달성했다. 하지만 타겟값 사이에서 산술 평균을 내버려 없거나 희미한 데이터를 예측했다.

K-Nearest Neighbor은 거리기반 알고리즘이기에 이웃 데이터의 가중치를 조정해 본 결과, Test R2가 0.5219를 기록했다.

6.2. 문제점

하이퍼 파라미터 튜닝과 파생 변수 생성 등 다각도의 노력에도 불구하고, 모든 회귀 모델의 예측 성능은 R2가 0.6을 넘지 못하는 벽을 보였다.

이유는 간단하다.

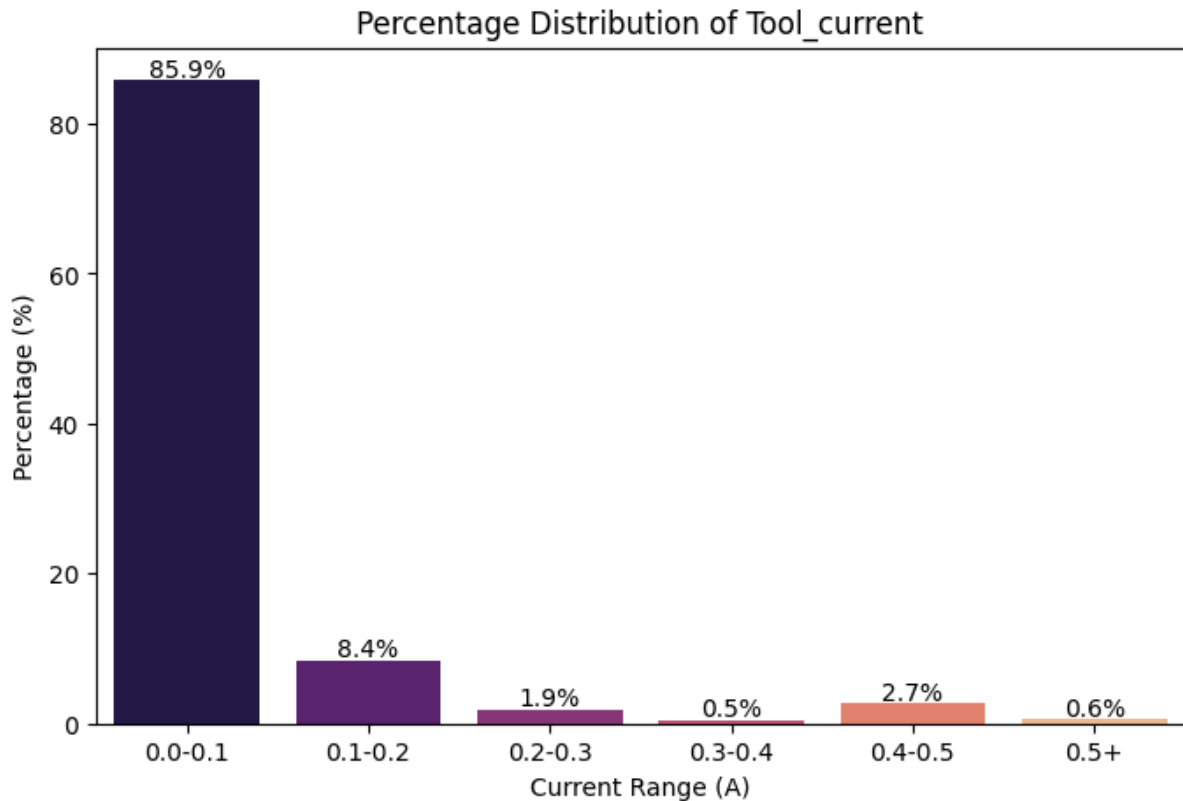


이 산점도 그래프는 Tool current 의 모든 분포 상태를 보여준다. 명확히 분리된 3개의 층을 형성하고 있다. 0.2A ~ 0.4A 사이의 흩어진 점들은 상태가 변하는 순간에 측정된 값이거나 노이즈일 것이다.

일반적인 회귀는 이 데이터들의 평균적인 위치를 지나도록 학습하므로, 데이터가 전혀 존재하지 않는 빈 공간을 관통하는 무의미한 예측선을 만들게 된다.

그리고 X 이 진행됨에 따라 값이 우상향하거나 우하향하는등의 추세가 없다. 다시말하면, 모든 구간에서 세가지 전류 상태가 동시에 불규칙하게 나타난다.

즉, 전류가 어떤 층에 위치할지 전혀 예측할 수 없다.



이 그래프는 각 분포를 %로 나타낸 그래프이다.

2절에서 분석했듯이 극심한 데이터 불균형이 나타나고있다.

해당 문제점들은 전류, 속도만으로는 과거의 파지 여부를 구분할 수 없는 물리적 한계를 증명하는 근거이며, 본 데이터셋이 연속형 회귀가 아닌 이산형 분류로 접근해야 함을 시사한다.

6.3. 결론, 향후 과제 및 Task의 패러다임 전환

비록 regression 관점에서 수행하기에 부적절한 주제였으나, 로봇팔의 전류와 속도 데이터만으로 그리퍼 전류 분산의 약 60%를 설명해냈다.

이는 로봇팔이 특정 자세를 취하거나 특정 속도로 기동할때, 그리퍼 역시 특정 작업을 수행하고 있음을 모델이 패턴화하여 학습했음을 의미한다.

선형 회귀에서 트리 및 이웃 기반 모델로 넘어가며 성능이 2배 이상 좋아진것은, 로봇 제어 데이터가 단순 비례 관계가 아닌 특정 조건에서 급변하는 고도의 비선형적 특성을 지니고 있음을 규명했다.

결론적으로, **Regression Task**를 수행하며 도출된 **R2 0.58**의 결과는 관절과 그리퍼간의 상관관계가 있음을 증명한 탐색 과정이었으며 그리퍼의 거동은 연속적인 수치 예측이 아닌 특정 상태를 식별하는 분류로 접근해야한다는 인사이트를 얻게 되었다.

따라서, 다른 주제의 **Classification Task**를 시작하기 전, 이 문제를 **Classification** 관점에서 다시 다루어보겠다.

Tool current를 대기, 전이, 파지로 라벨링하여 회귀모델이 겪었던 중간값 예측의 함정을 차단하고 각 물리적 상태를 얼마나 식별 해낼 수 있는지 검증하는것으로 분석을 전개하겠다.